

ABSTRAKT

Tato práce se zabývá návrhem a realizací digitální verze stolní hry Člověče, nezlob se, která umožňuje hru mezi skutečnými hráči a hráči řízenými algoritmem. Digitalizace stolních her umožňuje rozšíření možnosti interakce a přináší nové herní prvky využívající moderní technologie.

Cílem je implementace hrací desky založené na adresovatelném LED pásku WS2812B a jednočipovém počítači STM32. Hráči ovládají své figurky pomocí displeje s joystickem. Při hodu kostkou se zobrazí animace a následně si hráč vybere figurku, s níž se posune na herní desce.

Pro řízení systému je využit mikrokontroler STM32, který generuje signál PWM pro adresaci WS2812B. STM32 je využito i pro komunikaci s displejem a pamětí na rozšiřujícím modulu GFX01M2 pomocí SPI.

Výstupem je digitální verze stolní hry Člověče, nezlob se, která využívá STM32 pro řízení adresovatelného LED pásku a displeje. Úspěšně byla realizována interakce hráče s hrací deskou pomocí hodu kostky a výběru figurky. Práce ukazuje praktické využití mikrokontrolerů v interaktivních hrách. Díky propojení programování, hardwaru a herního designu může tento projekt usnadnit dětem a začátečníkům vstup do světa vestavěných systémů a elektroniky.

Obsah

Úvod	7
1. Blokové schéma	8
2. Individuálně adresovatelný RGB LED pásek	10
2.1 Interní struktura.....	10
2.2 Datový signál	11
2.3 Ovládání pomocí STM32	12
2.3.1 Časovač 1 TIM1	12
2.3.2 Přímý přístup do paměti DMA	15
2.3.3 Ovladač WS2812B.....	17
3. Displejový rozšiřující modul pro STM32	19
3.1 LCD displej	19
3.2 Flash paměť	22
3.3 Serial Peripheral Interface.....	25
3.3.1 Čtení z paměti SPI.....	26
3.4 TouchGFX	27
3.4.1 Obrazovka 1 & obrazovka 2 TouchGFX	28
3.4.2 Obrazovka 3 TouchGFX.....	30
3.4.3 Obrazovka 4 TouchGFX.....	31
3.4.4 Obrazovka 5 & obrazovka 6 TouchGFX	33
4. Deska plošných spojů	36
4.1 Schéma.....	36
4.1.1 Regulátor napětí LDO	36
4.1.2 Jednočipový počítač STM32	37
4.1.3 Rozšiřující modul s displejem X-NUCLEO-GFX01M2	39
4.1.4 Rozhraní pro Pixely WS2812B.....	40
4.2 Návrh desky plošných spojů	41
4.2.1 Verze 1 DPS.....	41
4.2.2 Verze 2 DPS.....	41
4.2.3 Verze 3 DPS.....	42
5. Vzniklé problémy	46

5.1	Pin NRST	46
5.2	Náhodné chování Pixelů	47
5.3	Ostatní problémy	48
5.3.1	Zkrat pinů SPI	48
5.3.2	Pájení ploch bez tepelných mostů	48
6.	Software pro ovládání hry	49
6.1	Pohyb figurek po hrací desce	49
6.2	Vyhození figurek	50
6.3	Dosažení cílového domečku	50
6.4	Animace v průběhu hry	51
6.4.1	Výběr figurky Animace	51
6.4.2	Nasazení a vyhození figurky Animace	51
6.4.3	Dosažení cíle Animace	52
6.4.4	Výhra hráče Animace	52
6.5	Nastavení Animace	52
6.6	Výběr figurky algoritmicky řízeného hráče	53
7.	Konstrukce	54
7.1	Popis konstrukce	54
7.2	3D tisk	54
Závěr		56
	Seznámení s WS2812B	56
	Hrací deska pro člověče nezlob se	56
	Možnost hry s algoritmickými hráči	56
	Návrh desky plošných spojů	57
	Návrhy na vylepšení	57
	Seznam použité literatury	58
	Seznam obrázků	60
	Přílohy	62

Úvod

Ve světě plném spěchu a stresu je někdy nejlepší si sednout ke stolu s rodinou nebo přáteli a užít si chvíli pohody. A právě desková hra Člověče, nezlob se je k tomu ideální. Tato klasická desková hra přináší nejen napětí a strategii, ale i spoustu smíchu a zábavy. Stačí jeden tah a váš soupeř se vrací na začátek. Hráči se ocitnou v prostředí plném rivality, překvapení a zábavy, což tuto hru činí nejen napínavou, ale i emotivně intenzivní.

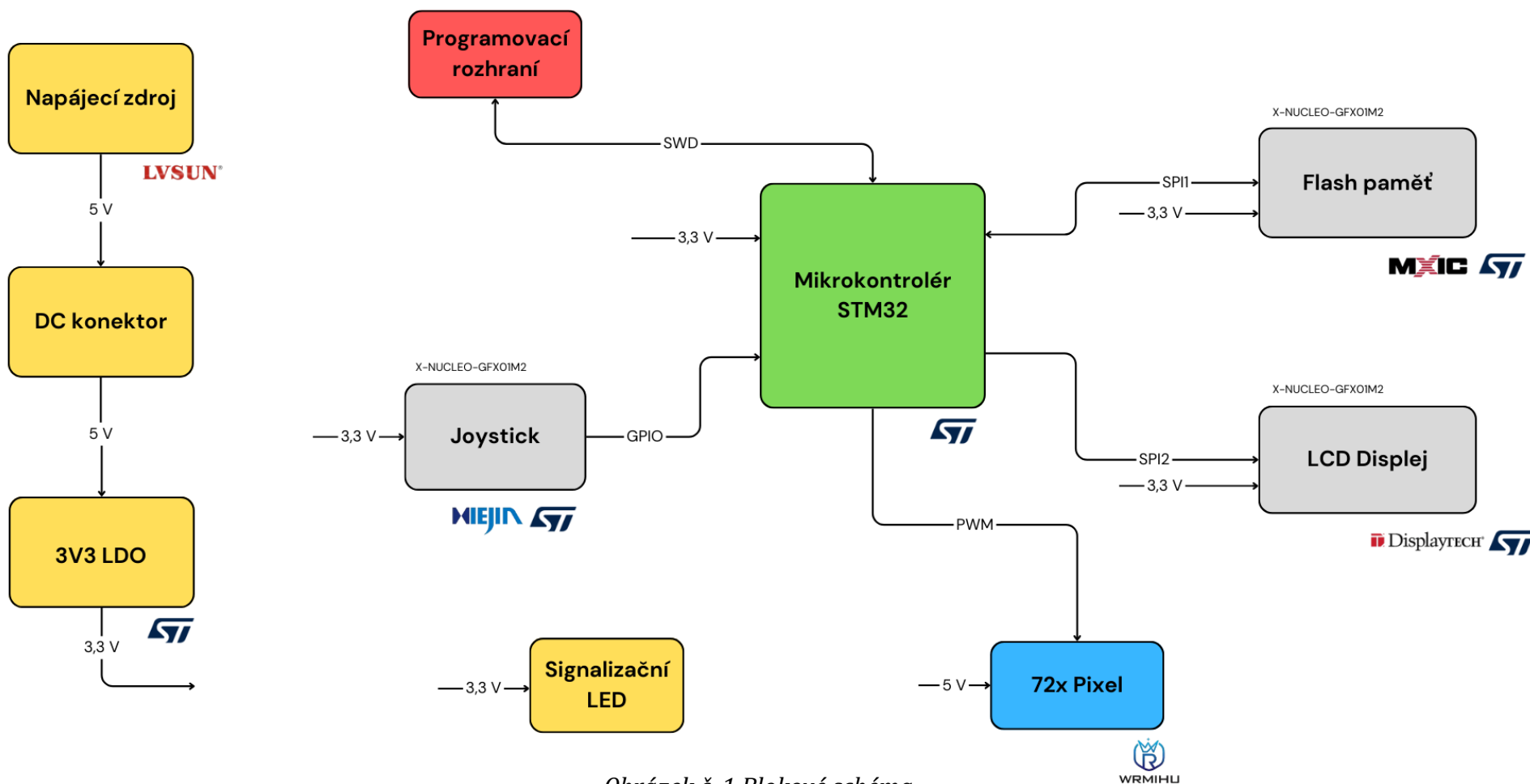
Pro zvelebení zážitku z této stolní hry se ji pokusím věrně převést do digitální formy. Hráčský zážitek bude obohacen o dynamické animace při nasazování a vyhazování figurek či při dosažení cílového domečku. Hráči si budou moci zvolit počet účastníků a případně povolit hráče řízené algoritmem, kteří mohou hrát společně se skutečnými hráči. Další příjemnou změnou oproti stolní verzi bude optimalizované grafické rozhraní s jednoduchou a intuitivní navigací pro interakci s figurkami.

Hrací deska je vytvořena ze sedmdesáti dvou barevných LED světýlek (Pixelů), které dynamicky reagují na průběh hry. Například při posunutí figurky se vypne Pixel, na kterém se figurka nacházela a rozsvítí se ten následující.

Pohyb joystickem slouží k výběru figurky a hodu kostkou, přičemž na displeji se zobrazí vybraná figurka a číslo, které hráč hodil. Tato vizuální zpětná vazba usnadňuje orientaci ve hře a umožňuje hráčům snadněji sledovat průběh a rozhodnutí v každém kole.

Digitální verze této známé deskové hry obsahuje několik upravených pravidel, která usnadňují hru a zároveň zvyšují úroveň strategie. Po dosažení cílového domečku není možné s figurkou dále pohybovat. To znamená, že všechny figurky, které se dostanou na první políčko cílového domečku, se automaticky posunou na konec domečku, čímž umožní vstup dalším figurkám. Na začátku hry má každý hráč pouze jeden hod. To znamená, že i když žádný hráč nemá nasazenou figurku, všichni hráči budou mít stále pouze jeden hod na kolo. Cílem těchto malých změn bylo povýšit úroveň strategie této hry a zároveň usnadnit přehlednost na hrací desce.

1. BLOKOVÉ SCHÉMA



Obrázek č. 1 Blokové schéma

Zdroj: Vlastní zpracování

Napájecí soustava je tvořena spínaným napájecím zdrojem *LS-PW36-5V4AV*, který je zapojen do zásuvky. Jeho druhý konec vede do DC konektoru, který se nachází na desce plošných spojů (DPS). Pro napájení jednočipového počítače je však potřeba napěťová úroveň 3,3 V, a proto je na DPS regulátor napětí, který nám převede 5 V na požadovanou úroveň. Jako kontrolka toho, že tato napájecí soustava správně funguje slouží signalizační LED.

Hlavním prvkem je jednočipový počítač STM32, který lze naprogramovat pomocí programovacího rozhraní SWD (Serial Wire Debug). Na toto rozhraní se napojí programátor a následně se do jednočipového počítače nahraje program.

Součástí firmwaru¹ je ovladač pro Pixely (WS2812B). Pixely tvoří hrací desku, po které se pohybují figurky jednotlivých hráčů. Rozhraní pro napojení této periferie se nachází na okraji DPS pro jednoduchý přístup.

Šedé bloky jsou součástí displejového rozšiřujícího modulu pro STM32. Joystick slouží jako rozhraní uživatele se zbytkem výrobku. Čtení a zápis do flash paměti probíhá pomocí SPI. Tato paměť je důležitá pro uchování grafických prvků pro displej, které by se nevešly do paměti, která je zabudovaná v jednočipovém počítači.

LCD TFT² displej je klíčovou součástí rozšiřujícího modulu, který slouží k zobrazení důležitých informací během hry. Díky němu hráči snadno vidí aktuální stav hry, jako například vybranou figurku nebo výsledky hodu kostkou. Displej také umožňuje přehledně nastavit parametry hry, jako je počet hráčů nebo zapnutí algoritmických hráčů.

Celý tento výrobek je navržen tak, aby byl intuitivní a snadno ovladatelný. Hráči interagují s hrací deskou pomocí joysticku, zatímco displej poskytuje jasné a přehledné informace.

¹ **Firmware** je software nainstalovaný na hardwarových zařízeních, který řídí jejich základní funkce a umožňuje komunikaci mezi komponenty.

² Podrobnosti o **LCD TFT** displeji budou uvedeny v kapitole LCD displej

2. INDIVIDUÁLNĚ ADRESOVATELNÝ RGB LED PÁSEK

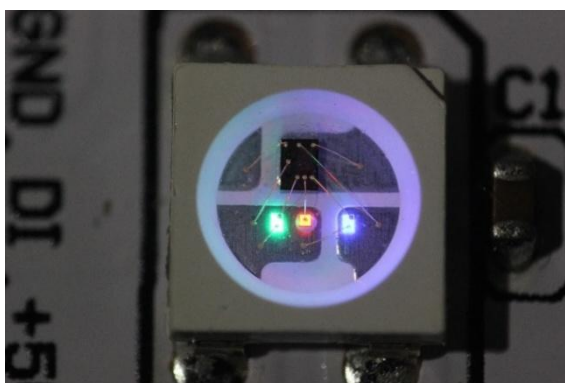
Hlavním prvkem hrací desky je individuálně adresovatelný RGB LED pásek WS2812B. Ten umožňuje libovolné nastavení barvy na každém ze 72 využitých Pixelů.

Řízení WS2812B probíhá pomocí speciálního časového protokolu, který je vygenerován pomocí PWM³ na STM32. Každý pixel přijímá 24b data určující barvy Red, Green a Blue. První Pixel v řetězci si přečte a uloží si prvních 24 bitů, zatímco zbývající data předává dále do řetězce.

2.1 INTERNÍ STRUKTURA

Každá WS2812B obsahuje svůj vlastní integrovaný obvod, který má tři hlavní úkoly:

- **Dekódování přijatých dat** – rozdělení přijatých 24 b na jednotlivé barevné složky Red, Green a Blue. Každá složka má svých 8 bitů což odpovídá v desítkové soustavě hodnotám 0–255
- **PWM signál** – na základě 8b barevných složek vygeneruje integrovaný obvod PWM signály, které ovládají jas kanálů R, G a B. Například: když červená barevná složka odpovídá hodnotě 255, tak integrovaný obvod vygeneruje PWM signál s DCL 100%
- **Poslání zbývajících dat** – po zpracování dat integrovaný obvod zbylá data zesílí a pošle dalšímu Pixelu v řetězci



Obrázek č. 2 WS2812B / Vnitřní struktura

Zdroj: spritesmods.com

³ **PWM** (Pulse Width Modulation) je technika umožňující řídit intenzitu signálu tím, že se mění poměr mezi dobou, kdy je signál aktivní (log. 1) a neaktivní (log. 0). Tento poměr se nazývá Duty Cycle (DLC) a určuje, jak dlouho je signál v aktivní fázi v rámci jedné periody

Každý Pixel provádí tyto operace pro zpravování signálu, který je v projektu generován pomocí jednočipového počítače STM32. V následujících kapitolách se podíváme na způsob generování tohoto signálu.

2.2 DATOVÝ SIGNÁL

Datový signál má několik požadavků, které musíme dodržet pro správný přenos dat. V této kapitole jsou uvedeny ty nejdůležitější.

Důležitým požadavkem datového protokolu pro pixely je časování, pro správné rozpoznání nízké a vysoké logické úrovně. V katalogu od výrobce jsou uvedeny tyto požadavky:

Data transfer time($T_H+T_L=1.25\mu s\pm 600ns$)

T0H	0 code ,high voltage time	0.4us	$\pm 150ns$
T1H	1 code ,high voltage time	0.85us	$\pm 150ns$
T0L	0 code , low voltage time	0.85us	$\pm 150ns$
T1L	1 code ,low voltage time	0.4us	$\pm 150ns$
RES	low voltage time	Above 50 μs	

Obrázek č. 3 WS2812B / Požadavky pro datový signál

Zdroj: shop.adafruit.com

Tabulka nám ukazuje časové hodnoty pro posílání logické jedničky a nuly. Tyto údaje jsou užitečné pro vývoj ovladače pro Pixely

Pro správné rozpoznání dat musí být dodržena velikost napětí datového signálu. V katalogu je uvedena tato podmínka:

$$\text{Input voltage level} = 0.7V_{DD} \quad (1)$$

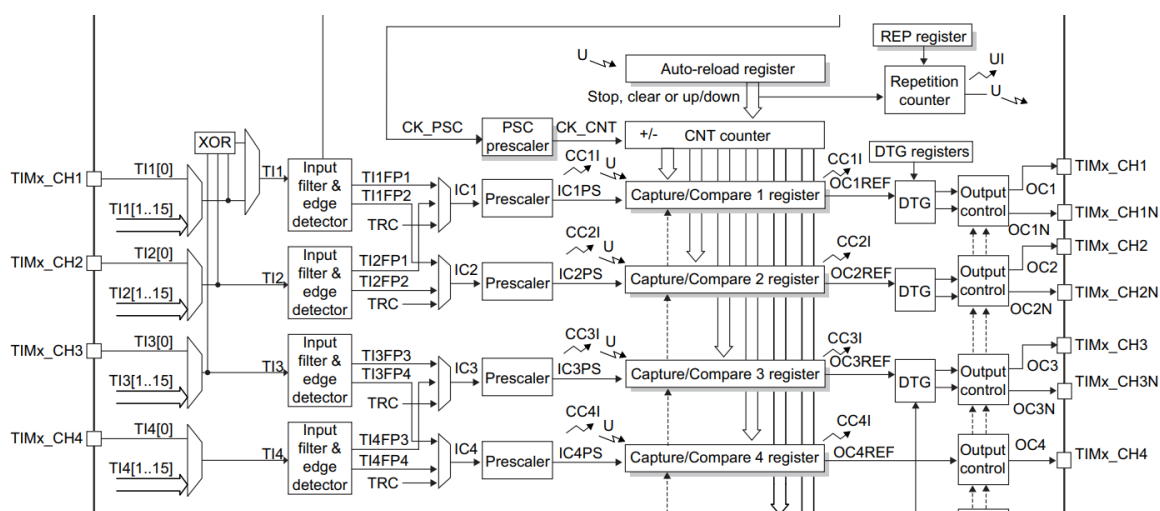
V_{DD} , tedy napájecí napětí je v našem případě 5 V. To znamená, že podle vzorce č. 1 by úroveň datového signálu měla být 3,5 V [15]. Výstupy STM32 využívají ale napětíovou úroveň 3,3 V. Teoreticky by data neměla být čitelná, ale opak je pravdou a Pixely jsou schopny číst tyto data. Samotný Pixel slouží jako převodník napětíové úrovně, který převede 3,3 V na požadovanou úroveň pomocí napájecího napětí.

2.3 OVLÁDÁNÍ POMOCÍ STM32

Pro ovládání Pixelů jsme využili jednočipový počítač STM32G071RBT6, který nabízí hned několik periférií, pomocí nichž je možné ovládat WS2812B. Pro jednoduchost jsme však využili periférii TIM1.

2.3.1 ČASOVAČ 1 | TIM1

Pro generování PWM je použit TIM1, který lze využít v různých režimech, přičemž v blokovém schématu vidíme, že každý kanál má svou vlastní vstupní cestu, která se využívá v režimu Capture.



Obrázek č. 4 TIM1 | Blokové schéma

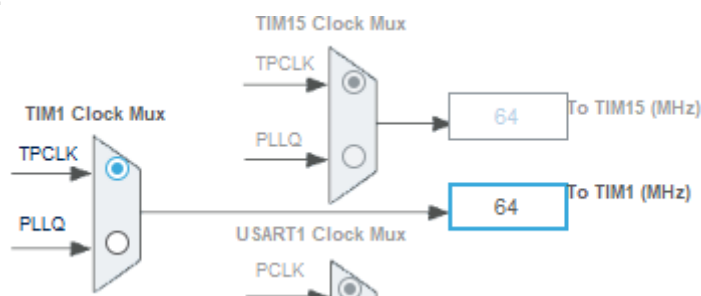
Zdroj: www.st.com

Signály $TI1[0]$, $TI1[1..15]$ představují vstupní zdroje pro kanál, což znamená, že každý kanál může reagovat na externí signál z více pinů, přičemž jejich výběr je řízen *multiplexerem*. Blok *XOR* (Exclusive OR) umožňuje kombinaci více vstupních signálů. Následující blok *Input filter & edge detector* signál vyfiltruje od šumu a umožňuje detekci náběžné, sestupné nebo obou hran. Vyfiltrovaný signál dále zpracovávají *IC1FP1*, *IC1FP2* a *Trigger Controller (TRC)*, které umožňují například synchronizaci na základě externí nebo interní události. Před vstupem do *Capture Compare registru* (CCR) prochází signál *prescalerem*, který může snížit frekvenci vstupního signálu. Díky tomu Capture režim umožňuje práci s externími signály bez zbytečného zatěžování CPU, protože veškeré zpracování probíhá v hardwaru.

Dále lze TIM1 konfigurovat do režimu Compare, který řídí výstupní vlnovou formu nebo indikuje, kdy uplynula určitá doba. Funkce používá čítač pro porovnání s hodnotou v registru CCR, a když dojde k shodě, může řídit výstupní pin různými způsoby, například přepínáním výstupu nebo jeho nastavením na vysokou nebo nízkou úroveň v závislosti na nastaveném režimu. To je užitečné pro generování přesných vlnových forem nebo časových signálů.

PWM režim je specializovaná forma režimu Compare, kde místo toho, aby došlo pouze k vyvolání události při shodě, neustále přepíná nebo řídí výstupní pin na základě hodnoty čítače.[12]

V předchozí kapitole jsme se dozvěděli, že perioda PWM signálu pro každý bit je 1,25 μ s. Na základě tohoto poznatku jsme nastavili časovač TIM1 v prostředí STM32CubeMX.



Obrázek č. 5 TIM1 / Nastavení hodin

Zdroj: Vlastní zpracování

Na obrázku vidíme, že TIM1 běží na frekvenci 64 MHz. Což odpovídá periodě 15,625 ns. Z těchto hodnot dopočítáme hodnotu *auto reload registru* (ARR) a následně *capture compare registru* (CCR).

$$ARR = \frac{T_{\text{žádané}}}{T_{CLK}} \quad (2)$$

$$ARR = \frac{1,25 \times 10^{-6}}{15,625 \times 10^{-9}} = 80$$

Jelikož ale čítání začíná od hodnoty 0 musíme skutečnou hodnotu upravit

$$ARR_{\text{skutečné}} = ARR - 1 \quad (3)$$

$$ARR_{\text{skutečné}} = 80 - 1 = 79$$

To tedy znamená, že hodnota CNT registru se inkrementuje každý tick⁴ časovače. Když se hodnota CNT rovná hodnotě v ARR vygeneruje se UEV (update event) a CNT se vyresetuje na hodnotu 0.

Když nastane UEV tak se nataví UIF (update interrupt flag) v registru TIM1_SR (status register). Když povolíme globální přerušení, tak tento flag může vyvolat přerušení, jehož rutina může provádět různé operace. Zatímco systém přerušení není klíčový pro generování PWM signálů, tak hraje roli při řízení neblokujících operací v systému.[12]

Dále musíme vypočítat hodnoty CCR tak, aby výstup odpovídal *Obrázek č. 3 WS2812B / Požadavky pro datový signál*. Pro generování PWM musíme nastavit všechny tři kanály, které používáme do PWM režimu. To zaručí, že když se hodnota CNT rovná hodnotě CCR dojde k přechodu pinu z hodnoty H do hodnoty L. Následně počkáme, než CNT dosáhne hodnoty ARR, čímž se nastaví pin zpět do hodnoty H.

Pro výpočet nás zajímají údaje T0H a T1H, jelikož určují čas v úrovni H při posílání T0 a T1.

$$CCR = \frac{T_H}{T_{clk}} \quad (4)$$

$$CCR_{T0} = \frac{400}{15.625} = 25,6 \cong 26$$

$$CCR_{T1} = \frac{800}{15,625} = 51,2 \cong 51$$

Díky toleranci ± 600 ns jsme schopni zaokrouhlit výsledek. Dalším důležitým údajem je pořadí, v jakém se mají jednotlivé barevné složky poslat. To můžeme opět najít v katalogu.

G7	G6	G5	G4	G3	G2	G1	G0	R7	R6	R5	R4	R3	R2	R1	R0	B7	B6	B5	B4	B3	B2	B1	B0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Note: Follow the order of GRB to sent data and the high bit sent at first.

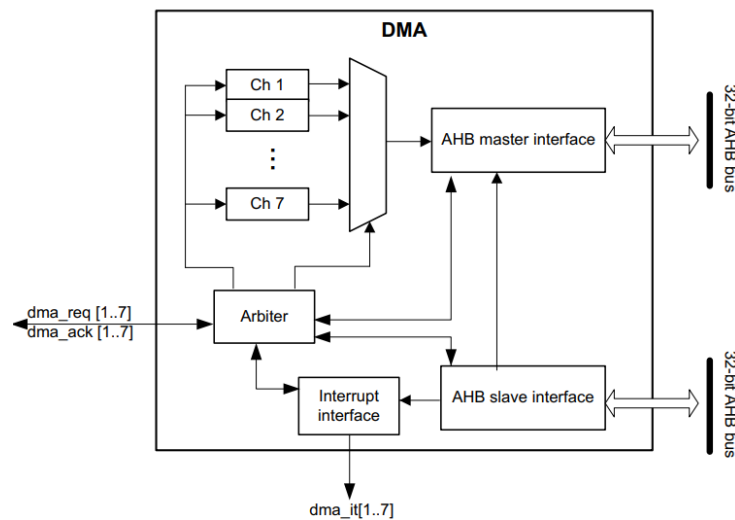
Obrázek č. 6 WS2812B / Uspořádání 24b informace

Zdroj: www.peace-corp.co.jp

⁴ **Tick** označuje jednotku času která odpovídá frekvenci při které timer funguje. V našem případě je to 64 MHz, tedy 15,6215 ns.

2.3.2 PŘÍMÝ PŘÍSTUP DO PAMĚTI | DMA

Abychom zbytečně nezatěžovali CPU, pro přenos dat využijeme DMA (Direct memory access). Tato periférie usnadňuje přenos dat mezi perifériemi a pamětí, čímž uvolníme CPU, aby mohlo provádět jiné operace. DMA využijeme pro přenos hodnot CCR_{T0} a CCR_{T1} z RAM přímo do registru $TIM1_CCR$. Tím dosáhneme toho, že jsme schopni ovládat Pixely a zároveň provádět jiné operace, na které je vyžadováno CPU.[12]



Obrázek č. 7 DMA / Blokový diagram DMA kontroléru

Zdroj: www.st.com

CH1, CH2, ..., CH7 reprezentují kanály DMA. Každý kanál může pracovat s jinými daty a přenášet je z/do RAM a periférií.

Blok *Arbiter* (rozhodce) rozhoduje jaký kanál dostane přístup k AHB sběrnici. Když máme více kanálů, které potřebují přenášet data ve stejnou chvíli, můžeme si u jednotlivých kanálů nastavit jejich prioritu. Tento blok rozhodne, jaký kanál bude moci přenášet svá data například na základě nastavené priority.

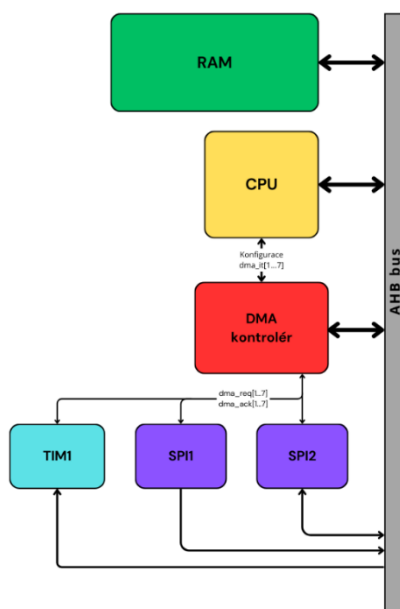
AHB master interface je rozhraní, přes které DMA kontrolér posílá data do paměti, nebo periférií. *AHB slave interface* je rozhraní, přes které DMA kontrolér přijímá data z paměti, nebo periférií.

32-bit AHB bus slouží jako přenosové médium po kterém proudí data v obou směrech (tedy z paměti k periférii a z periférie k paměti). Každý strojový cyklus je tato sběrnice schopna přenést až 32 b dat.

$dma_req[1...7]$ jsou požadavkové signály. Když chce periferie přenášet data pomocí DMA pošle požadavek (request) s informací po jakém kanále chce data přenášet (záleží na jaký kanál má daná periferie nastavené DMA)

signál $dma_ack[1...7]$ pošle DMA kontrolér zpět do periferie. Tím dá periférii vědět, že DMA kontrolér požadavek obdržel a může dojít k přenosu dat.

Po úspěšném přenosu dat nebo chybě pošle Interrupt Interface signál $dma_it[1...7]$ CPU. Tento signál je jediná interakce s CPU v celém procesu. CPU je závislé na těchto signálech, jelikož DMA a CPU nemohou zároveň číst nebo zapisovat z/do paměti ve stejnou chvíli. Tím, že CPU tento signál obdrží, si může být jistý, že může opět s pamětí pracovat.[12]



Obrázek č. 8 DMA / Blokové schéma využití DMA v systému

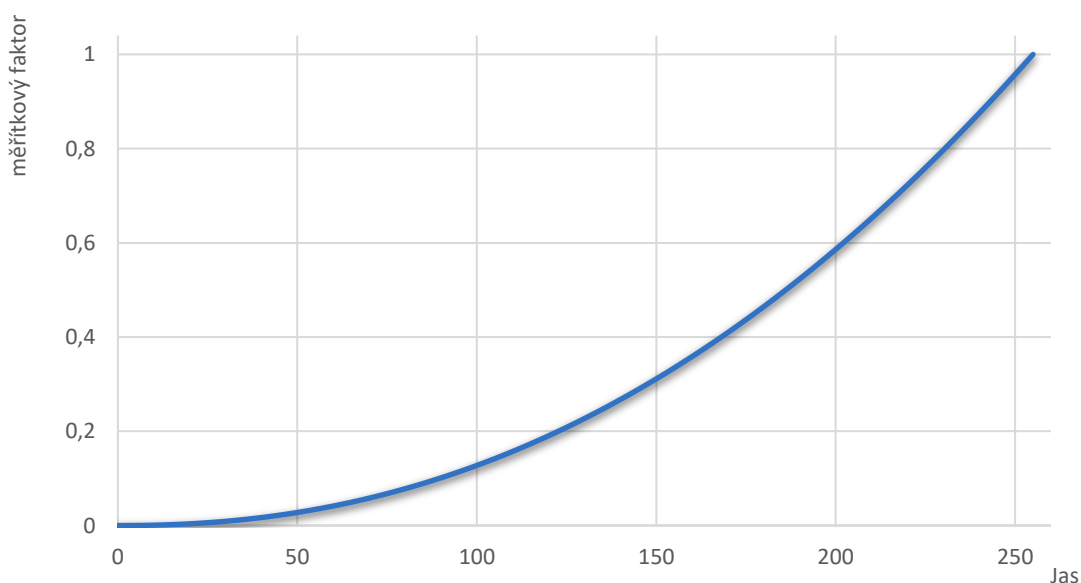
Zdroj: Vlastní zpracování

2.3.3 OVLADAČ | WS2812B

Dalším krokem je vytvoření driveru pro ovládání Pixelů. Ten je rozdělen na několik funkcí. Před tím, než se data pošlou je musíme vytvořit. Pro každý kanál TIM1 jsme vytvořili strukturu, která mimo jiné obsahuje dvojrozměrné pole, do kterého se ukládají jednotlivé barvy. Například do struktury korespondující s TIM1 CH1 zapíšeme do pole led_data na index 0 (1. Pixel) 255(R) 255(G) 0(B). To odpovídá žluté barvě.

Data můžeme dále zformátovat tím, že určíme jas, kterým bude tato barva vyzařována. Lidský zrakový systém reaguje logaritmicky, ne lineárně, což vede k schopnosti vnímat neuvěřitelný rozsah jasu (dynamický rozsah mezi scénami) přesahující 10 desetinných míst. [2] V praxi se mi nejvíce osvědčila následující mocninná funkce.

$$scale\ factor = \left(\frac{brightness}{255} \right)^{2,2} \quad (5)$$



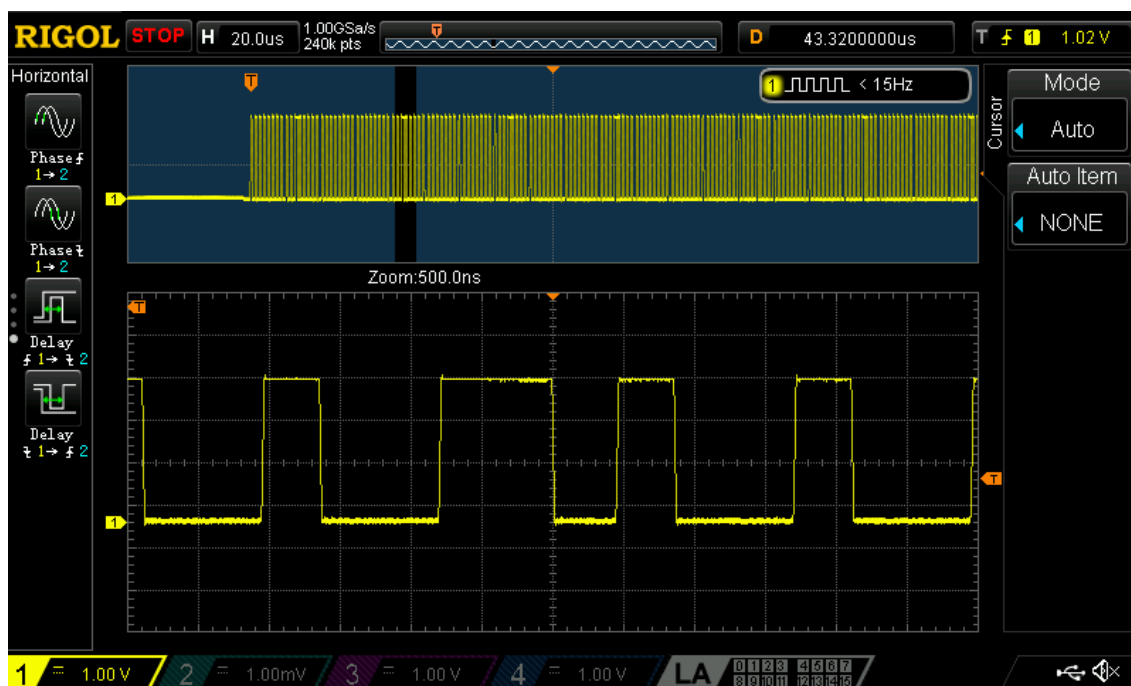
Obrázek č. 9 WS2812B | Graf měřítkového faktoru pro nastavení jasu

Zdroj: Vlastní zpracování

Měřítkový faktor (scale factor) se následně vynásobí s hodnotami R, G a B, čímž dosáhneme světla s požadovaným jasnem. Aby se měřítkový faktor nemusel vypočítávat pokaždé, když je nutné nastavit jas světla tak se měřítkový faktor

vypočítá pro všechny hodnoty jasu hned po zapnutí jednočipového počítače a uloží se do pole, z kterého následně pouze čteme.

Po nastavení jasu můžeme jednotlivé bity uspořádat do podoby GRB a následně pro každý bit s hodnotou 0 nastavíme do CCR hodnotu 26 a pro bit s hodnotou 1 do CCR registru vložíme hodnotu 51. Na konec řetězce přidáme resetovací pulz, který je delší než 50 μ s. Výsledný signál vypadá následovně:



Obrázek č. 10 WS2812B / Skutečný datový signál

Zdroj: Vlastní zpracování

3. DISPLEJOVÝ ROZŠÍŘUJÍCÍ MODUL PRO STM32

Rozhraní mezi hráči a hrací deskou je tvořeno pomocí displejového rozšiřujícího modulu pro vývojovou desku STM32 Nucleo s konektorem Morpho: X-NUCLEO-GFX01M2. Tento modul obsahuje 2.2" SPI QVGA⁵ TFT displej a 64-Mbit SPI NOR flash paměť pro ukládání grafických obrázků, textu a dalších prvků. Důležitou součástí rozšiřujícího modulu je také joystick pro navigaci. [11]



Obrázek č. 11 X-NUCLEO-GFX01M2

Zdroj: cz.rs-online.com

3.1 LCD DISPLEJ

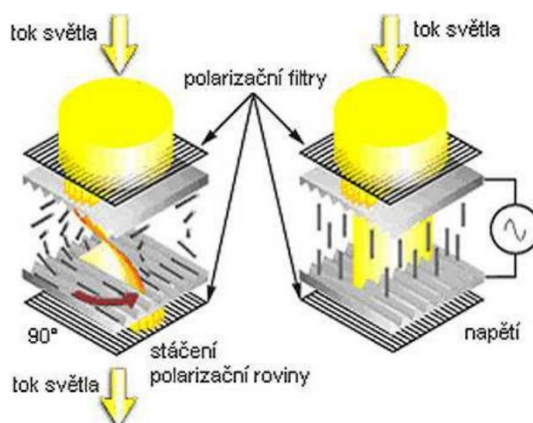
LCD (Liquid Crystal Display) displeje jsou složeny ze dvou polarizačních filtrů⁶, které jsou vůči sobě otočeny o 90°. Světlo, které projde jedním filtrem by tedy neprošlo tím druhým. Právě proto jsou mezi těmito dvěma filtry ještě 2 skleněné destičky mezi kterými jsou tekuté krystaly (Liquid Crystal).

Tyto krystaly jsou při vhodném uspořádání schopny změnit úhel, pod kterým letí vlna světla. Světlo projde, když jsou krystaly uspořádány do zakroucených útvarů (připomínajících točité schodiště) a tím otáčí vlnu procházejícího světla o 90° - tak aby světlo prošlo i druhým filtrem.

⁵ **QVGA** (Quarter Video Graphics Array) je grafický standard s rozlišením 320 × 240 pixelů, což je čtvrtina rozlišení VGA (640 × 480)

⁶ **Polarizační filtr** propustí pouze světlo, které je natočené pod určitým úhlem. Světlo můžeme vnímat jako vlny, nebo fotony. Světlo je složeno z mnoha vln. Polarizační filtr nám propustí pouze vlny, které jsou natočeny pod určitým úhlem.

Začne-li na krystaly působit elektrické pole, zorientují se krystaly v jeho směru (rovnoběžně), tím přestanou otáčet procházející světlo a druhý polarizační filtr ho nepropustí.

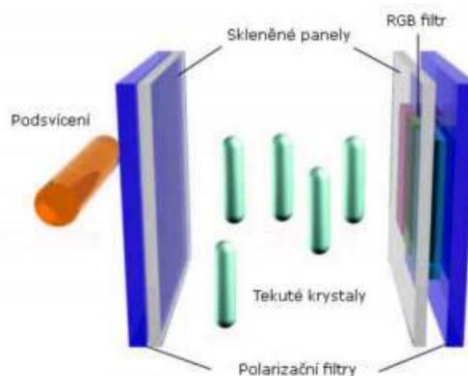


Obrázek č. 12 LCD / Princip funkce LCD

Zdroj: KRÍŽAN, V., EPO-11-Monitory. Olomouc

LCD displeje u kterých se tekuté krystaly uspořádají do tvaru spirálovitého schodiště bez působení elektrického pole se jmenují LCD Twisted Nematic (TN). Když na elektrody LCD TN displeje přivedeme napětí vznikne elektrické pole, které paralelizuje jednotlivé krystaly.

Tyto displeje světlo neumí vytvářet, ale pouze propouštět. To znamená, že světlo se na základě seskupení tekutých krystalů buď propustí, nebo ne. Pro správnou funkci je zapotřebí rovnoměrné osvětlení. To bývá v současnosti řešeno buď zářivkovou trubicí, nebo polem LED diod s bílým světlem (displeje označené jako „LED“).

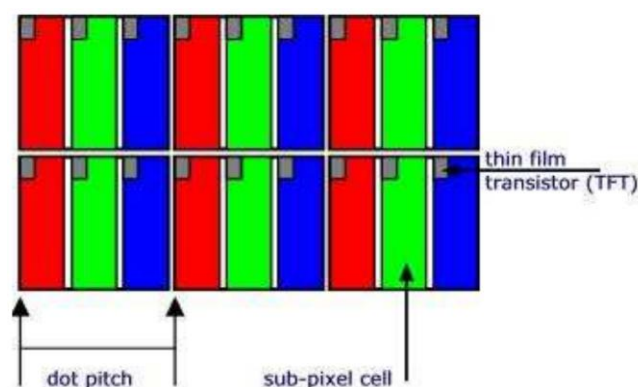


Obrázek č. 13 LCD / Konstrukce barevného LCD

Zdroj: KRÍŽAN, V., EPO-11-Monitory. Olomouc

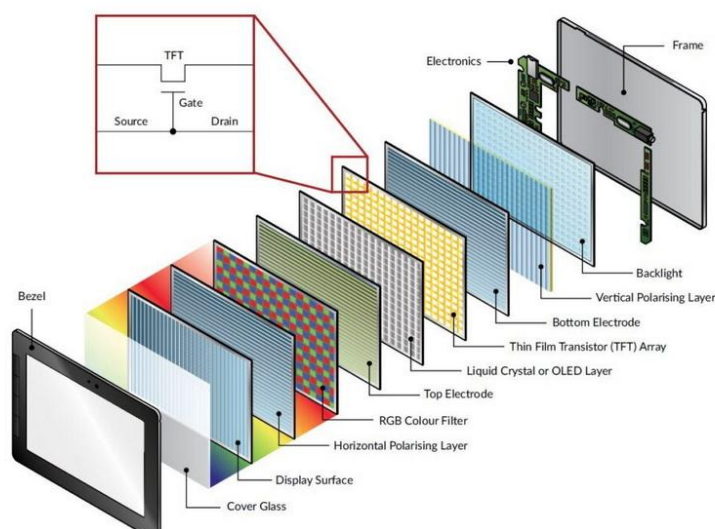
U barevných displejů je každý obrazový bod (pixel) složen ze tří samostatných bodů (tzv. subpixelů). Před každým subpixelem je umístěn filtr barvy R, G, nebo B. To stejné platí i pro spodní skleněnou destičku.

Abychom byli schopni ovládat kolik červené, zelené a modré barvy bude vyzařovat každý subpixel využívá se technologie TFT (Thin Film Transistor). LCD, které využívají této technologie se nazývají displeje s aktivním řízením. Každý subpixel obsahuje mikroskopický tranzistor typu FET, který řídí el. pole mnohem přesněji a pro jeho vytvoření je potřeba podstatně nižší napětí. Tranzistorová matice se přímo napařuje na skleněnou destičku s barevnými filtry technologií tenkých vrstev.[5]



Obrázek č. 14 LCD / Thin Film Transistor u jednotlivých subpixelů

Zdroj: KRÍŽAN, V., EPO-11-Monitory. Olomouc



Obrázek č. 15 LCD / Jednotlivé vrstvy displeje

Zdroj: www.reshine-display.com

3.2 FLASH PAMĚŤ

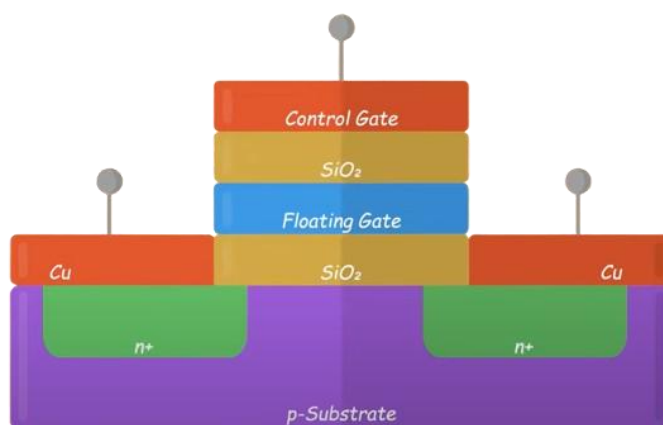
Flash paměť je druh elektronické paměti EEPROM⁷ (Electrically Erasable Programmable Read Only Memory), která je nevolatilní. To znamená že v ní data zůstanou uchována i po odpojení napájení. Dnes se flash paměť využívá v široké škále zařízení, například v mobilních telefonech nebo počítačích, kde slouží jako úložiště v podobě SSD (Solid State Drive).

Základním stavebním blokem tohoto druhu paměti je floating-gate MOSFET. Tento tranzistor funguje na stejném principu jako klasický MOSFET. Obsahuje ale několik změn – je obohacen o floating gate a control gate.

Floating gate slouží k uchování náboje. Tento náboj může v buňce zůstat nahaný několik desítek let. Porušením izolace jsme schopni náboj do floating gate nahrát. Tento proces se jmenuje tunelování a má destruktivní účinky – omezený počet zápisů.

Elektrony, které jsou uvnitř floating gate vytváří negativní potenciál, čímž se zvyšuje napětí, při kterém se tranzistor otevře. Když na control gate přivedeme malé napětí tranzistor zůstane zavřený – mezi source a drain neteče žádný proud (0).

Když ve floating gate žádný náboj není, napětí, při kterém se tranzistor otevře je nižší. To znamená, že když na control gate přivedeme stejné malé napětí tranzistor se otevře a mezi drain a source poteče malý proud (1).



Obrázek č. 16 Flash / Floating gate tranzistor

Zdroj: www.youtube.com

⁷ **EEPROM** znamená, že obsah paměti lze vymazat elektronickým signálem a následně znovu naprogramovat. Paměti typu EEPROM mají omezený počet cyklů. [9]

U tohoto druhu paměti však musíme řešit omezený počet zápisů. Řadič disku střídá umístění jednotlivých sektorů, čímž se snaží o co nejrovnoměrnější vytížení všech paměťových bloků.

Počet zápisů závisí na typu paměťové buňky. Existuje několik variant, včetně SLC (Single-Level Cell), která uchovává 1 bit informace, MLC (Multi-Level Cell) s 2 bity, TLC (Triple-Level Cell), QLC (Quad-Level Cell) a PLC (Penta-Level Cell).

Například u MLC místo jednoduchého nabíjení nebo vybíjení floating gate, jako u SLC, lze do ní vytunelovat jednu ze čtyř předem definovaných úrovní náboje. Každá z těchto úrovní odpovídá určitému rozsahu napětí, které je potřeba aplikovat na control gate při čtení, aby bylo možné správně rozlišit uloženou hodnotu.

Tabulka č. 1 Flash / Stavová tabulka SLC

Zdroj: Vlastní zpracování

U_T [V]	SLC
0	0
3	1

Tabulka č. 2 Flash / Stavová tabulka MLC

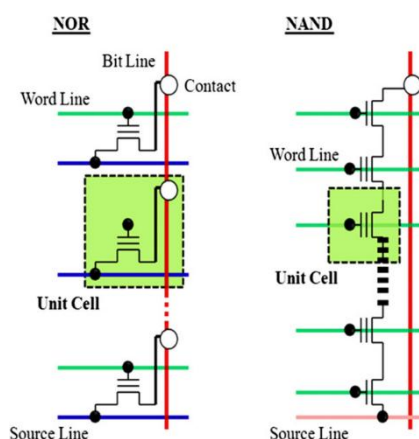
Zdroj: Vlastní zpracování

U_T [V]	MLC	
0	0	0
1	0	1
2	1	0
3	1	1

Stejná logika platí i pro TLC, QLC a PLC. Hodnoty U_T jsou pouze orientační. Tato technologie má své výhody i nevýhody. Mezi hlavní výhody patří: nízká spotřeba a vysoká rychlost (záleží na druhu operace). Tento druh paměti je také odolný vůči otřesům a magnetickému poli. Hlavní nevýhodou je omezený počet zápisů, což vede ke kratší živostnosti.[3][4]

Existují dva druhy této nevolatilní paměti NOR Flash a NAND Flash. U NOR Flash je každá paměťová buňka připojena paralelně mezi datový vodič (Bit Line) a source vodič. Control gate každé buňky je pak propojen s adresovým vodičem (Word Line). Tato architektura umožňuje přímý přístup k jednotlivým buňkám, což usnadňuje čtení a provádění náhodného přístupu k datům. Zapojení připomíná logickou funkci NOR.

U NAND Flash jsou paměťové buňky zapojeny sériově mezi datovým vodičem a source vodičem, což snižuje počet vodičů a zvyšuje hustotu paměti. Control gate každé buňky je opět připojen k adresovému vodiči (Word Line). Díky tomuto uspořádání je NAND Flash kompaktnější a rychlejší při sekvenčním čtení a zápisu, ale na rozdíl od NOR Flash neumožňuje přímý přístup k jednotlivým buňkám – čtení a zápis probíhá v blocích. Zapojení připomíná logickou funkci NAND. [7]



Obrázek č. 17 Flash | NOR vs. NAND

Zdroj: www.researchgate.net

Na rozšiřujícím modulu pro STM32 je využita NOR flash s kapacitou 64 Mbit. V katalogovém listu je uvedeno, že čip využívá proprietární buňky, ale s největší pravděpodobností můžeme odhadnout, že paměti tohoto typu využívají SLC, jelikož vyžadujeme spolehlivost a poměrně vysokou rychlost čtení.[11][6]



Obrázek č. 18 Flash | MX25L6433FM2I-08G

Zdroj: www.mouser.de

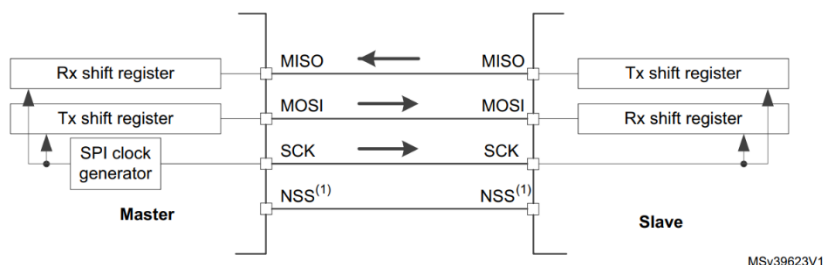
3.3 SERIAL PERIPHERAL INTERFACE

Sériové periferní rozhraní (SPI) je synchronní sériové komunikační rozhraní určené pro krátké vzdálenosti, zejména ve vestavěných systémech. Rozhraní bylo vyvinuto společností Motorola v polovině 80. let a stalo se de facto standardem.

Zařízení SPI mohou komunikovat v plně duplexním režimu pomocí architektury master-slave (full - duplex), přičemž nejčastěji se používá jeden master. Master (řídící zařízení) vytváří komunikační rámec pro čtení a zápis. Podpora více slave zařízení je možná pomocí individuálních čipových výběrových linek (Chip Select, CS), někdy nazývaných Slave Select (SS).

V plně duplexním režimu s více slave zařízeními SPI využívá čtyři sběrnice:

- SCK (Serial Clock) – hodinová sběrnice řízená masterem
- MISO (Master Input Slave Output) – data z slave zařízení do masteru
- MOSI (Master Output Slave Input) – data z masteru do slave zařízení
- SS (Slave Select), někdy CS (Chip Select) – linka pro výběr konkrétního slave zařízení ve víceslave konfiguraci (na následujícím obrázku označeno jako NSS) [12]

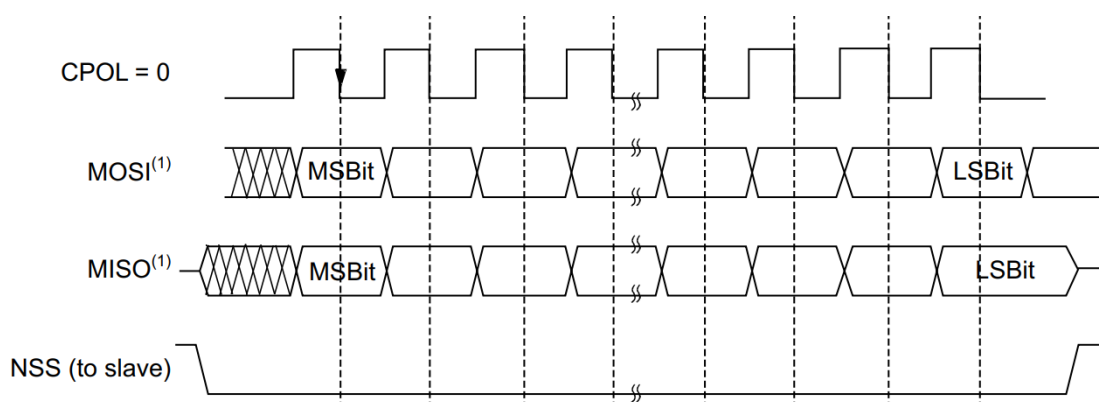


Obrázek č. 19 SPI / Full-duplex komunikace

Zdroj: www.st.com

Ve výchozím nastavení je SPI nakonfigurováno pro komunikaci v plném duplexu. V této konfiguraci jsou posuvné registry mastera a slava propojeny pomocí dvou jednosměrných linek mezi piny MOSI a MISO. Během SPI komunikace jsou data posouvána synchronně na hranách hodinového signálu SCK poskytovaného masterem. Master přenáší data, která mají být odeslána slavu, přes linku MOSI a přijímá data od slavu přes linku MISO. Jakmile je přenos datového rámce dokončen (všechna data jsou posunuta), dochází k výměně informací mezi masterem a slavem.

Tento režim je využit pro komunikaci s flash pamětí. SPI lze nastavit i do dalších režimů jako Half duplex, Receive only a Transmit only. [12]



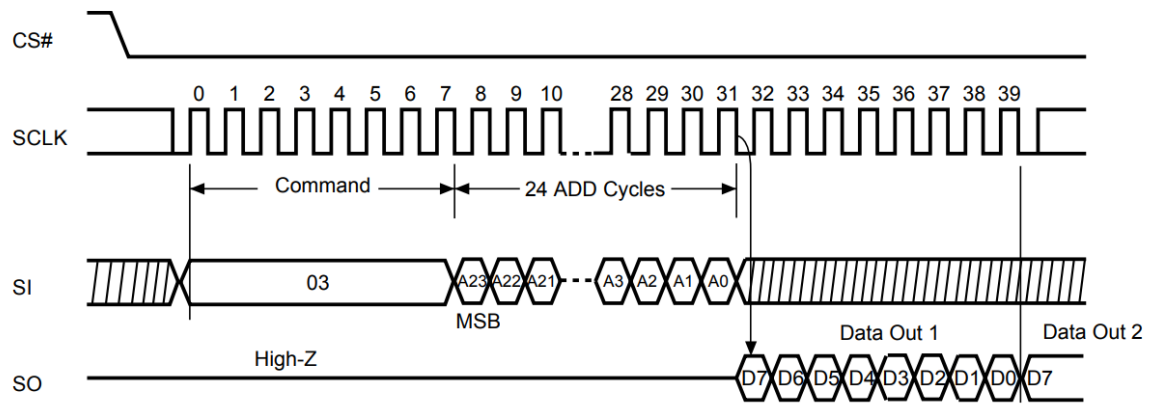
Obrázek č. 20 SPI | Diagram toku dat

Zdroj: www.st.com

Signál popsaný jako CPOL = 0 představuje SCK (Serial Clock). Nastavením parametru CPOL určujeme polaritu vzorkování signálu MOSI a MISO. Pro CPOL = 0 platí, že na nástupné hraně signálu SCK dochází ke změně hodnoty bitu na MOSI nebo MISO, zatímco při sestupné hraně SCK probíhá vzorkování těchto signálů. Přenos dat lze začít detekovat, když se signál CS (NSS) nastaví na logickou nulu (low). Po dokončení přenosu dat se signál CS opět nastaví na logickou jedničku (high), čímž se přenos ukončí. [12] Následující obrázek ukazuje tento proces, při čtení z flash paměti MX25L6433FM2I-08G.

3.3.1 ČTENÍ Z PAMĚTI | SPI

Instrukce pro čtení slouží k načítání dat. Adresa je zachycena při vzestupné hraně SCLK a data se posouvají na sestupné hraně SCLK. První adresa může být na libovolné pozici. Adresa je automaticky zvýšena na následující vyšší adresu po každém vyčtení datového bajtu, takže celá paměť může být přečtena pomocí jediné instrukce READ. Po dosažení nejvyšší adresy se adresní čítač přetočí na 0.



Obrázek č. 21 SPI | Čtení z flash paměti

Zdroj: eu.mouser.com

Sekvence provedení instrukce READ je následující: CS# se nastaví na nízkou úroveň, odešle se kód instrukce READ (Command) a 3B adresa na SI – MOSI (Adresa, od které začne čtení) přečtená data se posílají na SO – MISO. Pro ukončení operace READ lze použít CS#, který se kdykoli během přenosu dat nastaví na vysokou úroveň. [6]

3.4 TOUCHGFX

Tento balíček obsahuje vše potřebné pro kompletní implementaci grafického uživatelského rozhraní (GUI) na hardware založeném na STM32.

TouchGFX se skládá ze tří hlavních částí – dvou nástrojů a jednoho frameworku:

TouchGFX Designer je intuitivní nástroj pro návrh GUI v TouchGFX, který umožňuje snadno vytvářet vizuální vzhled aplikace.

TouchGFX Generator je plugin pro STM32CubeMX, ve kterém lze konfigurovat a generovat vlastní **TouchGFX Abstraction Layer (AL)** pro konkrétní hardware založený na STM32.

TouchGFX Engine je C++ framework, který pohání UI aplikaci. Zajišťuje aktualizaci obrazovky, zpracování uživatelských událostí a správu časování. Technologie TouchGFX je optimalizována pro mikrokontroléry STM32, čímž poskytuje maximální výkon při minimálním zatížení CPU a nízké spotřebě paměti.[14]



Obrázek č. 22 TouchGFX / Logo

Zdroj: support.touchgfx.com

Návrh GUI proběhl v aplikaci TouchGFX Designer. Tato aplikace vygeneruje kód pro jednotlivé objekty. Následně se musí nainstalovat TouchGFX Engine. To je možné provést z prostředí STM32CubeMX. Vygenerovaný kód následně můžeme otestovat tím, že ho nahrajeme do jednočipového počítače STM32. Implementace logiky pro jednotlivé objekty se musí naprogramovat například v prostředí STM32CubeIDE. Popis této logiky je popsán v následujících odstavcích.

3.4.1 OBRAZOVKA 1 & OBRAZOVKA 2 | TOUCHGFX

Grafické rozhraní v projektu je složeno ze šesti různých obrazovek (načítání, nastavení, hod kostkou, výběr figurky, výsledky hry a další statistiky). Tyto obrazovky jsou pojmenovány Screen1 – Screen6 a tak budou dále označovány.

Screen1 slouží jako načítací obrazovka při přechodu mezi ostatními obrazovkami. Její využití je minimální, a proto se ní nebudeme zabývat.

Screen2 slouží k nastavení před začátkem hry. Hráči si mohou vybrat celkový počet hráčů, zapnout hráče řízené algoritmem a případně počet skutečných i algoritmických hráčů.

Pro realizaci je využito vlastních kontejnerů (TouchGFX Designer). Po vytvoření vlastního kontejneru se vytvoří C++ třída, která dědí z třídy *Container*.



Obrázek č. 23 TouchGFX / numberOfAllPlayersSetting

Zdroj: Vlastní zpracování

Třída *Screen2View* obsahuje instanci třídy (objekt) **numberOfAllPlayersSetting**. Třída *Screen2View* dále obsahuje několik metod, pro ovládání logiky přechodu mezi jednotlivými instancemi vlastních kontejnerů a jejich logiku.

Třída *Screen2View* obsahuje mimo jiné dvě metody: *incrementValue* a *decrementValue*. První jmenovaná metoda se zavolá, když hráč joystickem pohne doprava a druhá, když se pohne joystickem doleva. Na základě právě zvoleného widgetu (vlastního kontejneru) se zavolá *handle* metoda. V případě *numberOfAllPlayersSetting* se zavolá metoda *handleNumberOfAllPlayersSetting*. Tato metoda obsahuje logiku pro ovládání widgetu.

Tato *handle* metoda vyžaduje argument *handle_type*. Dle tohoto argumentu určíme, jestli se počet hráčů přičte, nebo odečte (definováno pomocí enumerace). Když je tedy *handle_type INCREMENT*, inkrementujeme počet hráčů a zkontrolujeme, jestli počet hráčů nepřesahuje 4. Když ano, tak počet hráčů nastavíme na 2. Objekt *numberOfPlayersSetting1* obsahuje metodu *changeNumberOfPlayers*, kterou zavoláme, aby se změna počtu hráčů provedla vizuálně na samotném displeji. Dále nastavíme proměnnou *delay_event* na hodnotu *ARROW_RIGHT_ANIMATION*, uložíme si aktuální tick a schováme šipku, která ukazuje doprava. Metoda *handleTickEvent* třídy *Screen2View* zařídí, že se šipka opět na displeji zobrazí za 15 ticků, tedy za 250 ms (1 s je 60 ticků). Jako poslední nám zbývá změnit barvu kruhu, který zobrazuje barvy hrajících hráčů. To zařídí metoda *handleColorCircles*, která je součástí objektu *numberOfPlayersSetting1*.

Tato metoda přijímá argument určující počet hráčů. Po jejím zavolání uložíme do pole ukazatele na objekty *Circle*. Toto pole obsahuje čtyři ukazatele, z nichž každý odkazuje na instanci třídy *Circle*. Následně iterujeme všemi čtyřmi objekty a nastavíme jejich viditelnost na false. Potom pomocí smyčky do while nastavíme jejich viditelnost na True na základě počtu hráčů. Výsledkem je widget na obrázku č.23.

Tím, že joystickem pohneme nahoru nebo dolů změníme možnost nastavení (kontejner). Když tedy pohneme dolů zobrazí se nám widget **fillWithBotsSetting**. Tento widget obsahuje přepínací tlačítko, pomocí kterého si hráči zvolí, zda si přejí, aby mohli hrát s algoritmi řízenými hráči. Metoda *handleFillWithBots* vyžaduje argument *button_state*, který nám říká, jestli uživatel tlačítko zapnul nebo vypnul. Když je hodnota proměnné *button_state* *BUTTON_OFF*, nastavíme počet skutečných hráčů na počet celkových hráčů (nutné pro prevenci těžké chyby) a počet

algoritmicky řízených hráčů na 0. Následně zavoláme metodu *setButtonState* objektu *fillWithBotsSetting1*, která nastaví vizuální podobu tlačítka.

Tím že pohneme joystickem dolů, se nám zobrazí další možnost nastavení, a to widget *numberOfRealPlayersSetting*. Tento widget funguje na stejném principu jako *numberOfAllPlayersSetting* a slouží k nastavení počtu skutečných hráčů.

Poslední možností v nastavení je widget *numberOfAIPlayersSetting*, ke kterému se hráč dostane pohybem joysticku dolů. Princip funkce je stejný jako u *numberOfRealPlayersSetting* a *numberOfAllPlayersSetting*. Hráči si pomocí tohoto widgetu mohou zvolit počet algoritmicky řízených hráčů.

Pro začátek hry se musí hráči dostat pomocí pohybu joystickem nahoru a zmáčknout středové tlačítko.



Obrázek č. 24 TouchGFX | Widgety *fillWithBotsSetting*, *numberOfAIPlayersSetting*, *numberOfRealPlayersSetting* a *startGame*

Zdroj: Vlastní zpracování

3.4.2 OBRAZOVKA 3 | TOUCHGFX

Screen3 zobrazuje animaci házení kostkou. Ta je provedena pomocí widgetu *Animated Image*. Ten prochází devíti obrázky hrací kostky v nekonečné smyčce. Barva pozadí je závislá na právě hrajícím hráči. Například po zapnutí hry hraje jako první červený hráč, pozadí obrazovky je tedy červené.

Vždy, když se na displeji zobrazí tato obrazovka, zavolá se metoda *setupScreen* objektu *Screen3View*. V této metodě je kód, který se vykoná při načtení obrazovky. Součástí této metody je zavolání funkce *setBoxColor*. Ta vyžaduje dva argumenty - referenci na objekt *Box* (pozadí) a číslo hráče, který právě hraje (*player*). Následně se nastaví barva objektu *Box* podle argumentu *player*.

Dále musíme zjistit, jestli je hráč algoritmicky řízený, když ano tak se v pravém horním rohu displeje zobrazí nápis AI a po jedné vteřině se zobrazí číslo, které hráč hodil. Když hráč není algoritmicky řízený, tak musí zmáčknout středové tlačítko joysticku, následně se na displeji zobrazí hozené číslo a po jedné vteřině displej automaticky přejde na obrazovku 4.



Obrázek č. 26 TouchGFX / Obrazovka 3

Zdroj: Vlastní zpracování



Obrázek č. 25 TouchGFX / Výsledek hodu kostkou

Zdroj: stock.adobe.com

3.4.3 OBRAZOVKA 4 | TOUCHGFX

Tato obrazovka se zobrazí po hodu kostkou a slouží k výběru figurky. V metodě *setupScreen* nejprve zjistíme, zda hráč, který právě hraje, již dokončil hru a dostal všechny čtyři své figurky do cíle. Pokud ano, hráč se přeskočí (stejné pravidlo platí i pro obrazovku 3). Dále si uložíme strukturu právě hrajícího hráče, která obsahuje všechny potřebné informace. Nastavíme také pozadí obrazovky pomocí funkce *setBoxColor* a v levém horním rohu zobrazíme hrací kostku odpovídající hozenému číslu.

Při inicializaci se automaticky vybere první dostupná figurka. Pokud je však již v cíli, zvýrazní se následující figurka. V takovém případě také aktualizujeme číslo

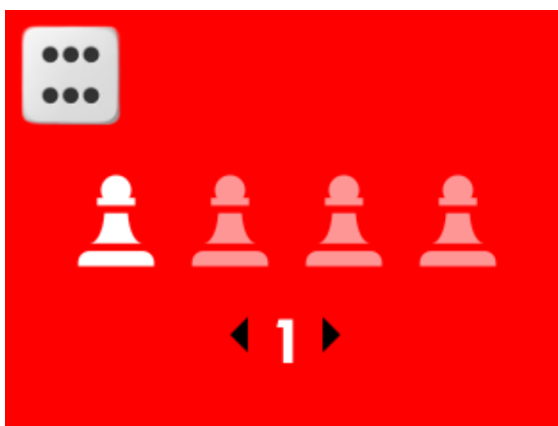
zobrazené pod figurkami. Dále je nutné graficky upravit průhlednost jednotlivých figurek, což provádí metoda *setPawnAlpha*.

Metoda *setPawnAlpha* přijímá argument *pawn* (vybraná figurka). Do pole uložíme čtyři ukazatele na objekty *Image* (figurky 1–4) a postupně projdeme jejich pozice. Pokud figurka není v cíli, nastavíme její průhlednost. Pokud je *pawn* rovna aktuálně iterované figurce, nastavíme její průhlednost na 255 (plně viditelná), jinak na 150 (částečně průhledná).

Dále musíme zjistit, zda je hráč řízen algoritmem. Pokud ano, zavoláme funkci, která automaticky vybere figurku (podrobnosti o algoritmu jsou v kapitole 6.6 Výběr figurky algoritmicky řízeného hráče). Na základě vybrané figurky znovu nastavíme průhlednost (*setPawnAlpha*) a aktualizujeme číslo pod figurkami. V pravém horním rohu se zobrazí text AI. Aby skutečný hráč nemohl výběr měnit, zablokujeme joystick.

Skutečný hráč vybírá figurku pohybem joysticku doleva nebo doprava. Pokud se pokusí přesunout za poslední figurku, výběr se vrátí na první figurku, a naopak – při pohybu před první figurku se ocitne zpět na poslední. Pohyb joysticku spustí animaci odpovídající šipky (doleva nebo doprava), která po krátké chvíli zmizí a za 400 ms se znovu objeví. Když si hráč je jistý svým výběrem, stisknutím středového tlačítka joysticku jej potvrdí.

Následně se přesměrujeme zpět na obrazovku 3, kde další hráč hází kostkou. Tato smyčka se opakuje, dokud na hracím poli nezůstane pouze jeden hráč. V takovém případě hra končí a zobrazí se obrazovka 5.



Obrázek č. 27 TouchGFX | Obrazovka 4

Zdroj: Vlastní zpracování

3.4.4 OBRAZOVKA 5 & OBRAZOVKA 6 | TOUCHGFX

Po ukončení hry se zobrazí **obrazovka 5**, která obsahuje **konečné pořadí hráčů**. Toto je řešeno v metodě `setupScreen` třídy `Screen5View`.

Pro určení pořadí iterujeme přes všechny hráče a ukládáme si jejich strukturu. Tato struktura obsahuje proměnnou `is_finished`, která se nastaví na **true**, pokud hráč dostal všechny své figurky do cíle. Pokud při iteraci narazíme na hráče s `!is_finished`, víme, že je poslední.

Pořadí hráčů ukládáme do struktury `game_info`, konkrétně do pole `results`. Každému hráči přiřadíme jeho pozici tak, že na index (výsledná pozice - 1) uložíme jeho číslo. Tyto výsledky musíme zobrazit graficky. Pro tento účel jsme vytvořili další vlastní kontejner, který se jmenuje `playerResultContainer`. Jeho vzhled vypadá následovně:



Obrázek č. 28 TouchGFX | playerResultContainer

Zdroj: Vlastní zpracování

Do pole uložíme čtyři ukazatele na instance třídy `playerResultContainer`. Poté pomocí smyčky a podmínky nastavíme viditelnost těchto objektů podle počtu hráčů. Následuje další iterace přes jednotlivé objekty. Do lokální proměnné `player` uložíme číslo hráče z pole `results`. Například při první iteraci, kdy `container = 0`, získáme číslo hráče z `results[0]`. Z pole `result_containers` vybereme odpovídající objekt podle indexu a zavoláme na něj metodu `setResultPositionTextAndColor`. Tato metoda vyžaduje dva argumenty – číslo hráče a jeho výslednou pozici.

Tato metoda změní vizuálně číslo a nastaví barvu kruhu. To se provede pomocí funkce `setCircleColor`, která vyžaduje tři argumenty. `circle painter`⁸ objekt, `circle` objekt a `player`. Následně se podle hodnoty argumentu `player` nastaví barva kruhu.

⁸ Pro tvorbu kruhu pomocí TouchGFX musíme využít více objektů. Objekt `Circle` definuje tvar kruhu, zatímco objekt `PainterRGB565` (`circle painter`) definuje to, jak je kruh vyplněn.

V pravém dolním rohu této obrazovky vidíme animovanou ikonu joysticku, která nás nabádá pohnout joystickem směrem dolů. Animace je řešena stejným principem, jako animace šipek na předchozí obrazovce.



Obrázek č. 29 TouchGFX / Obrazovka 5

Zdroj: Vlastní zpracování

K přechodu na obrazovku číslo 6 musíme joystickem pohnout směrem dolů. Tato obrazovka zobrazuje doplňující informace o hře jmenovitě:

- Celkový počet hodů
- Počet nasazených figurek
- Počet vyhozených figurek
- Celková doba hry
- Čas dokončení prvního hráče
- Čas dokončení druhého hráče
- Čas dokončení třetího hráče.

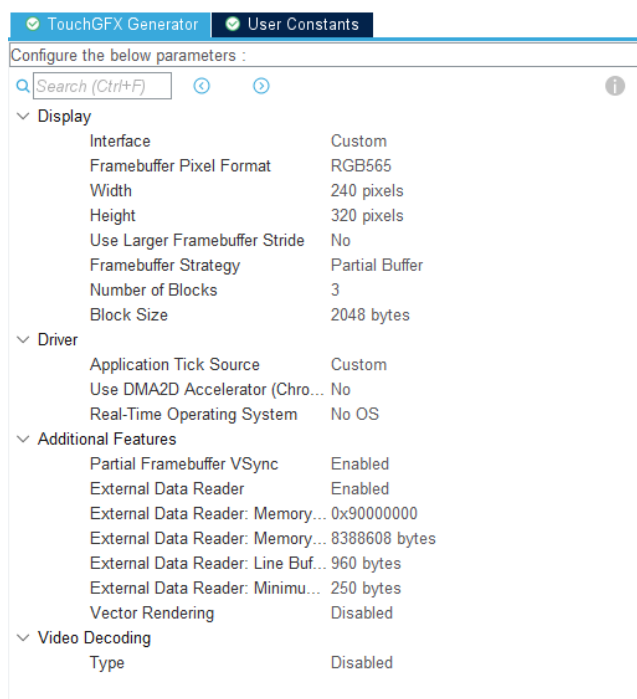
Poslední dva body se zobrazí jen tehdy, když je počet hráčů vyšší než dva. Součástí Screen6View je metoda setGameStats, která se volá v metodě setupScreen. Tato metoda nastaví již zmíněné statistiky. Například celková doba hry je měřena pomocí časovače TIM7, který je nastaven tak, aby vyvolal přerušení každou milisekundu. Součástí callback funkce TIM7 je inkrementace proměnné miliseconds. Když hra skončí, uloží se hodnota této proměnné a následně je přepočítána na minuty a sekundy, které se zobrazí na displeji. Stejný postup je aplikován i u ostatních statistik.

V pravém dolním rohu opět vidíme dva animované joysticky, které nám ukazují, že pohybem nahoru se dostaneme zpět na obrazovku 5 a stisknutím středového tlačítka se vyresetuje celá hra a jsme přesunuti zpět do nastavení.



Obrázek č. 31 TouchGFX / Obrazovka 6

Zdroj: Vlastní zpracování



Obrázek č. 30 TouchGFX / Nastavení TouchGFX Generator

Zdroj: Vlastní zpracování

4. DESKA PLOŠNÝCH SPOJŮ

Deska plošných spojů (DPS) je základní konstrukční prvek elektronického zařízení a integruje veškeré komponenty nutné pro jeho správnou funkci. Obsahuje prvky sloužící k regulaci napájecího napětí, řízení externích periférií a komunikaci s dalšími moduly, včetně displeje a externí paměti.

4.1 SCHÉMA

Stejně jako DPS i schéma prošlo několika iteracemi. Mezi nejvýznamnější změny finální verze patří DC konektor pro napájení, zjednodušená filtrace napětí a rozhraní pro reproduktor. Schéma také obsahuje čtyři montážní otvory, které jsou důležité pro uchycení ke konstrukci. Dále bylo opraveno několik chyb v rozhraní pro rozšiřující modul s displejem. Následující kapitoly popisují důležité části DPS.

4.1.1 REGULÁTOR NAPĚTÍ | LDO

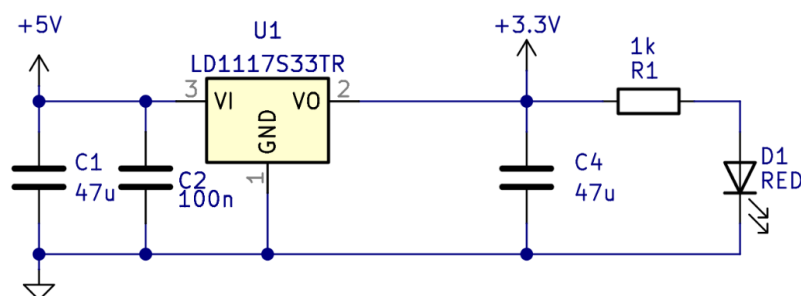
Pro stabilní napájení obvodu je použit nízkonapěťový lineární regulátor LD1117S33TR od společnosti STMicroelectronics, který zajišťuje konverzi napětí z 5 V na 3,3 V. Tato napěťová úroveň je nezbytná pro správnou funkci mikrokontroleru STM32 a připojeného displeje.

Regulátor LD1117S33TR je nízkopoklesový (LDO – Low Dropout) stabilizátor, což znamená, že ke správné regulaci vyžaduje jen malý rozdíl mezi vstupním a výstupním napětím⁹. To jej činí vhodným pro převod 5 V na 3,3 V, přičemž minimalizuje ztrátový výkon a zahřívání součástky.[9]

Pro zajištění stability a potlačení případného šumu jsou na vstupu a výstupu použity kondenzátory. C1 na vstupu slouží k potlačení napěťových špiček a kompenzaci při náhlých změnách odběru. Kondenzátor C2 slouží k eliminaci vysokofrekvenčního rušení. Na výstupu se nachází kondenzátor C4, který slouží ke stabilizaci výstupního napětí, zajišťuje správnou funkci regulátoru a snižuje případné kolísání napětí při změnách zátěže.

⁹ Hodnoty poklesů napětí (U_d dropout voltage) a jeho velikost je závislá na velikost protékajícího proudu. Pro $I_0 = 100 \text{ mA}$ je $U_d = 1 \text{ V} - 1,1 \text{ V}$, pro $I_0 = 500 \text{ mA}$ je $U_d = 1,05 \text{ V} - 1,15 \text{ V}$ a pro $I_0 = 800 \text{ mA}$ je úbytek napětí na LDO $U_d = 1,1 \text{ V} - 1,2 \text{ V}$. [9]

Popsané zapojení bylo mírně upraveno oproti doporučenému schématu výrobce s cílem zlepšit filtraci.



Obrázek č. 32 LDO / Regulátor napětí

Zdroj: Vlastní zpracování

4.1.2 JEDNOČIPOVÝ POČÍTAČ | STM32

Zapojení jednočipového počítače STM32 je zobrazeno na následujícím obrázku. Piny PC14, PC15 a PF0, PF1 nejsou k ničemu napojené. Tyto piny jdou využít k připojení externího oscilátoru (HSE a LSE) v projektu je tedy využito interních oscilátorů (HSI RC a LSI RC), které však nejsou tak přesné, ale jsou dostačující.

V_{DD} , tedy napájecí napětí jednočipového počítače se může pohybovat mezi 1,7 (1,6) – 3,6 V. V_{DD} je externí napájecí zdroj pro interní regulátor a systémovou analogovou část, jako je reset, správa napájení a interní hodiny. Je poskytováno externě přes pin VDD/VDDA.

V_{DDA} je analogový napájecí zdroj pro A/D převodník, D/A převodník, vyhlazovací buffer napětí a komparátory. Úroveň napětí V_{DDA} je identická s napětím V_{DD} , protože je poskytováno externě přes pin VDD/VDDA.

V_{BAT} je napájecí zdroj pro RTC¹⁰, TAMP¹¹, nízko rychlostní externí oscilátor 32,768 kHz a záložní registry, když není přítomno napětí V_{DD} . V_{BAT} je poskytováno externě přes pin VBAT.

V_{REF+} je referenční napětí pro analogové periferie nebo výstup interního vyhlazovacího napěťového bufferu (pokud je povolen).[8]

¹⁰ **RTC (Real-Time Clock)** – je speciální hardware v mikrokontroleru, který slouží k udržování přesného času a datumu, i když je hlavní napájení vypnuto.

¹¹ **TAMP (Tamper Detection)** je funkce mikrokontroleru STM32 určená k detekci neoprávněné manipulace se zařízením, zejména se záložními registry a RTC.

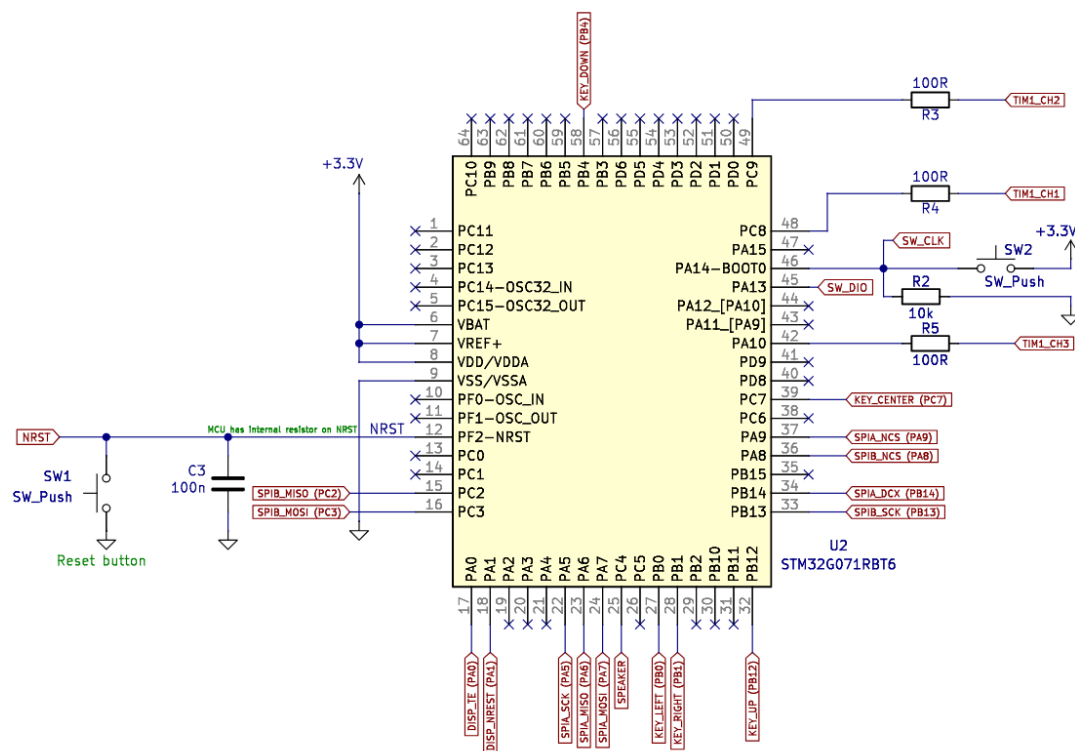
Piny VBAT, VREF+ a VDD/VDDA jsou v zapojení propojeny a jde do nich napěťová úroveň 3,3 V. Pin VSS je propojen se zemnicí plochou (Označeno GND).

PF2 – NRST slouží k restartování celého jednočipového počítače. Tento pin obsahuje interní pull-up rezistor. To znamená, že v klidovém stavu se pin nachází v úrovni VDD. Externě máme k pinu připojený filtrovací kondenzátor, který slouží jako zkrat pro vysokofrekvenční signály (šum, nebo rychlé přechody). Tlačítko, které se nachází mezi zemí a NRST slouží k propojení těchto dvou cest, čímž se vyresetuje STM32. Po puštění tlačítka se pin bude nacházet zpět na úrovni VDD. Funkce pinu NRST byla v průběhu debugování odstraněna (viz. kapitola 5.1)

Pin PA14 – BOOT0 má dva účely. Tento pin určuje režim, ve kterém bude jednočipový počítač pracovat po zapnutí. Když je na pinu log. 0 (propojeno se zemí) STM32 se spustí z flash paměti. To znamená, že bude vykonávat program, který je v ní nahrán. Když se pin nachází v log. 1(propojeno s VDD) znamená to, že se STM32 pokusí nastartovat ze systémové paměti. [8]

Tam se nachází vestavěný bootloader. Bootloader může být využit například k programování flash paměti různými komunikačními protokoly jako USART nebo USB nebo pro aktualizaci firmwaru. My ale potřebujeme, aby se vykonával program, který je nahrán ve flash paměti, a proto musíme pin držet v nízké logické úrovni to je provedeno pomocí pull-down rezistoru R2 s hodnotou 10 kΩ.

Druhým využitím tohoto pinu je SW_CLK, což je jeden z pinů potřebný pro debugovací rozhraní SWD (Serial Wire Debug). Toto rozhraní nám umožňuje čtení a zapisování z/do paměti, provádění programu po jednotlivých instrukcích a možnost bodů přerušení (breakpoint). Komunikace s programátorem je provedena pomocí pěti pinů a to PA14 (SW_CLK), PA13 (SW_DIO), PF2 (NRST), VDD a VSS. Program se do jednočipového počítače nahraje pomocí ST-LINK/V2.



Obrázek č. 33 STM32 | Zapojení jednočipového počítače

Zdroj: Vlastní zpracování

4.1.3 ROZŠÍŘUJÍCÍ MODUL S DISPLEJEM | X-NUCLEO-GFX01M2

Rozhraní mezi mikrokontrolerem a displejem je realizováno pomocí kolíkové lišty. Zapojení na následujícím obrázku je navrženo v souladu se schématem rozšiřujícího modulu, přičemž jednotlivé signálové vodiče jsou připojeny na konkrétní periferie mikrokontroleru.

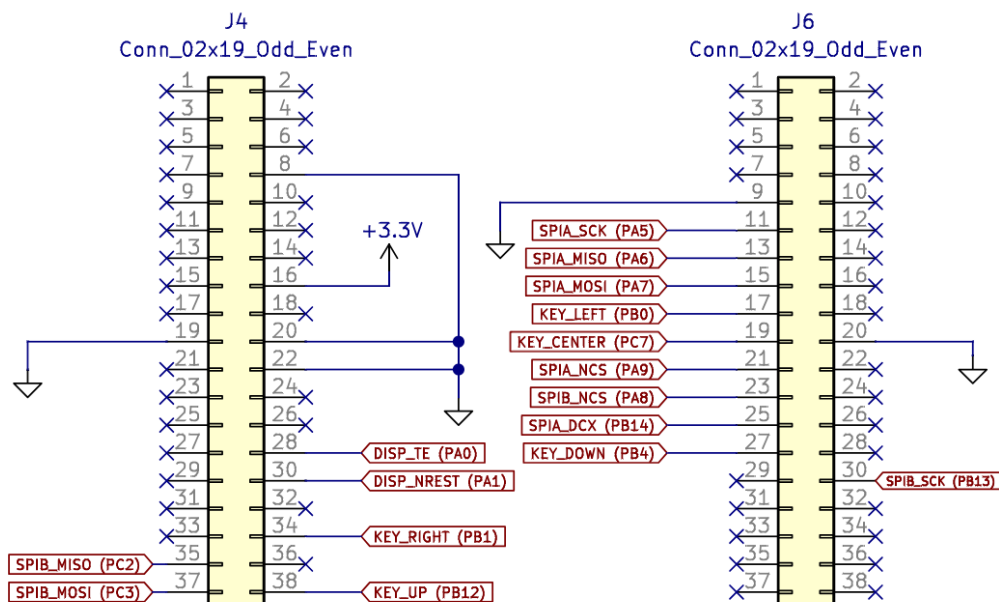
Komunikace s displejem i dalšími periferiemi probíhá prostřednictvím SPI sběrnice, přičemž existují dvě samostatné instance:

SPI1 je vyhrazeno pro komunikaci s LCD displejem. Je nastaveno do half-duplex režimu, což znamená, že využívá pouze CS, SCLK a MOSI vodiče, zatímco MISO není potřeba. SPI2 slouží ke komunikaci s flash pamětí a je nakonfigurováno v full-duplex režimu, takže využívá všechny čtyři vodiče (CS, SCLK, MOSI a MISO).

Vodiče označené jako KEY_X jsou signálové linky propojené s joystickem, který slouží k ovládání systému.

Rozšiřující modul je propojen se zemnicí plochou na několika místech, což pomáhá minimalizovat odpor zemnění a zajišťuje spolehlivou návratovou cestu signálů. Toto

opatření přispívá ke zlepšení elektromagnetické kompatibility (EMC) a snižuje možné rušení, které by mohlo ovlivnit stabilitu komunikace mezi jednotlivými komponentami.



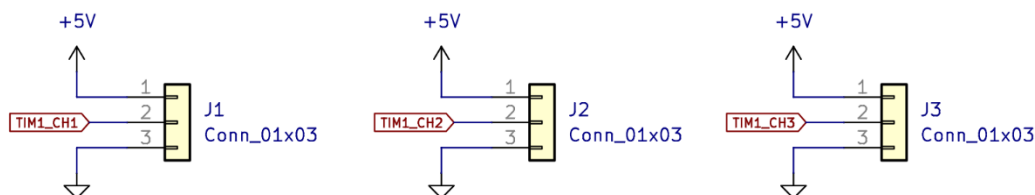
Obrázek č. 34 X-NUCLEO-GFX01M2 | Rozhraní pro rozšiřující modul

Zdroj: Vlastní zpracování

4.1.4 ROZHRAŇÍ PRO PIXELY | WS2812B

Rozhraní pro LED pásek je realizováno pomocí kolíkových lišt. Každý pásek vyžaduje připojení ke zdroji napětí 5 V, zemnímu vodiči (GND) a datovému signálu, který je generován na pinech TIM1_CH1 – TIM1_CH3 prostřednictvím PWM.

Pro ochranu jednočipového počítače je každý pin generující PWM signál opatřen rezistorem, jehož hlavním úkolem je omezit proud tekoucí do Pixelů a zabránit poškození GPIO jednočipového počítače. Za ideálních podmínek by sice nemusel být nutný, ale v případě zkratu může výrazně snížit riziko přetížení a následného zničení pinu. Hodnota rezistoru není kritická, a proto je využito rezistoru s hodnotou 100 Ω .



Obrázek č. 35 Rozhraní pro připojení LED pásků

Zdroj: Vlastní zpracování

4.2 NÁVRH DESKY PLOŠNÝCH SPOJŮ

Návrh DPS prošel několika iteracemi, které vedly k optimalizaci rozmístění součástek, zlepšení signálové integrity a minimalizaci rušení. Při vývoji byl kladen důraz na správné dimenzování napájecích cest, zajištění dostatečného odvodu tepla a dodržení doporučených konstrukčních pravidel pro vysokofrekvenční signály.

4.2.1 VERZE 1 | DPS

Původní verze desky plošných spojů obsahovala několik nadbytečných komponent, zejména v oblasti filtrace napájení VDDA. Protože však systém nevyužívá ADC ani DAC, nebylo nutné dodatečně filtrovat vstupní napětí.

Na DPS byly osazeny dva externí krystaly, připojené k pinům LSE a HSE jednočipového počítače. Konstrukční nedostatky se projeví zejména v oblasti filtrace napájecího napětí, která nebyla optimálně navržena – napěťový regulátor byl umístěn příliš daleko od vstupu napájení, což vedlo k nežádoucímu poklesu napětí a potenciálnímu rušení.

Deska měla čtyři vrstvy, přičemž dvě vnitřní byly vyhrazeny pro GND a +5 V. Ukázalo se však, že toto řešení bylo zbytečné, protože veškeré spoje bylo možné efektivně umístit pouze na dvě vrstvy. Další nevýhodou první verze DPS byla absence resetovacího tlačítka připojeného k pinu NRST.

4.2.2 VERZE 2 | DPS

Ve druhé revizi jsme odstranili několik již zmíněných problémů. Mezi hlavní vylepšení této verze patří resetovací tlačítko připojené na pin NRST, rozhraní pro reproduktor a signalizační LED indikující napájení.

Přesto měl návrh stále několik nedostatků. Například některé součástky byly osazeny na obou stranách desky, což nebylo optimální vzhledem k mechanické konstrukci. Dále obsahovala zbytečně dlouhé napájecí cesty, což mohlo vést k poklesu napětí a rušení. Regulátor stále nebyl ideálně umístěn pro efektivní napájení.

Napájecí cesty byly oproti první verzi několikanásobně širší (2 mm), což umožnilo vyšší proudovou zatížitelnost. I přes tyto změny však deska stále nebyla plně

optimalizovaná pro EMC (Electromagnetic Compatibility) a EMI (Electromagnetic interference), což mohlo negativně ovlivnit elektromagnetickou kompatibilitu. Celá DPS byla přepracována tak, aby se vešla na dvě vrstvy namísto původních čtyř.

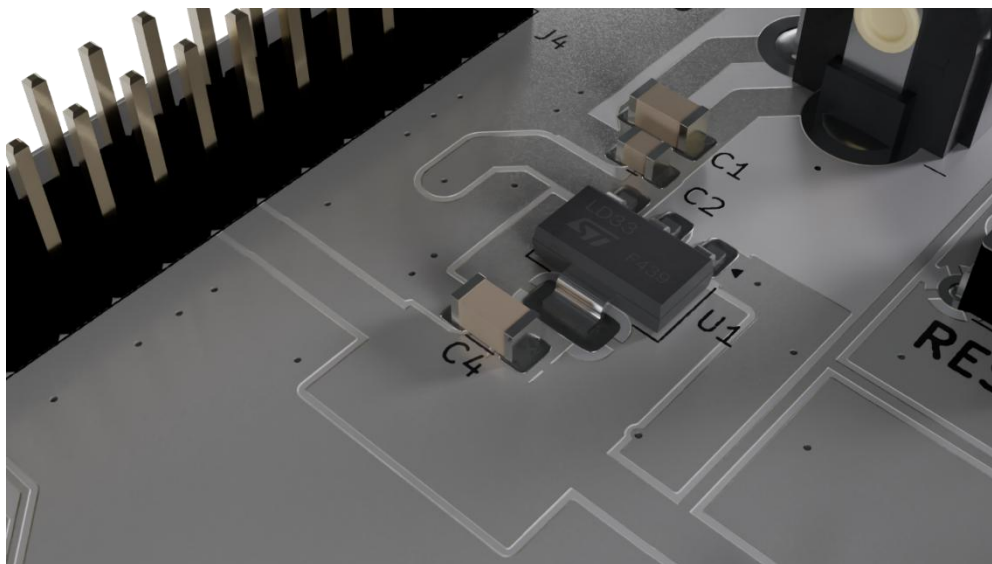
4.2.3 VERZE 3 | DPS

Třetí a finální verze opět opravila mnoho předchozích chyb a přinesla novinky pro lepší kompatibilitu.

Signálové trasy mají šířku 0,3 (0,5) mm. Napájecí cesty mají šířku 2 mm. Široké napájecí cesty mají několik důležitých vlastností jako je možnost vyššího toku proudu díky sníženému odporu širších cest nebo lepší rozptýlení tepla, které vzniká při vyšších proudech. Dále široké cesty přispívají k menšímu úbytku napětí na cestě nebo snížení problémů s EMC jako šum, či EMI. Širší cesty tedy umožňují hladký tok proudu, čímž se vyhneme problémům.

Obě vrstvy F.Cu a B.Cu slouží jako zemní roviny. Vrstvy jsou propojeny prokovy.

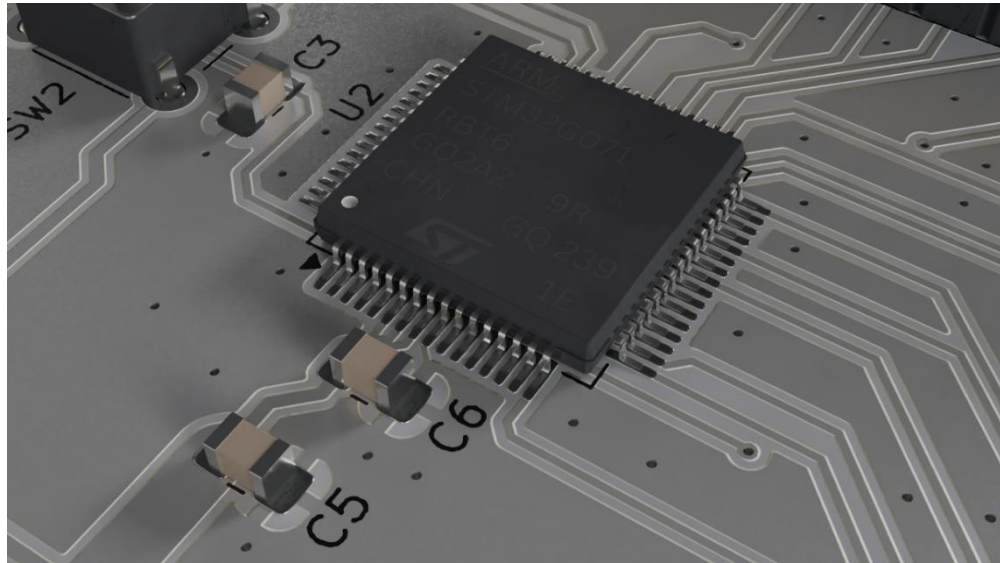
Kolem regulátoru napětí se nachází takzvaná termální podložka, která slouží k lepšímu rozptýlení tepla na výstupu regulátoru. Stejného principu je využito u tranzistoru U3.



Obrázek č. 36 DPS | Termální podložka u regulátoru napětí

Zdroj: Vlastní zpracování

Rozložení komponent je mnohem kompaktnější než v předchozích verzích. Filtrovací kondenzátory se nachází v malé vzdálenosti od regulátoru. Nejbližší ke vstupu vidíme kondenzátor s hodnotou 100 nF (C2), za ním (C1) 47 μ F a kousek od výstupu kondenzátor 47 μ F (C4). Vrchní napájecí cesta (5 V) se po filtraci prokovem dostane na druhou stranu desky a vede až ke kolíkové liště, která slouží jako rozhraní pro WS2812B. Z výstupu LDO je napěťová úroveň 3,3 V napojena krátkou širokou cestou k jednočipovému počítači a rozšiřující desce.

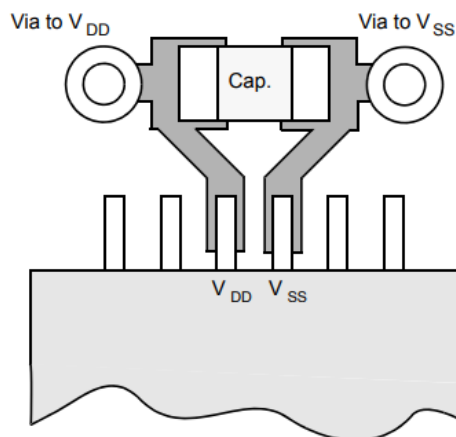


Obrázek č. 37 DPS | Rozložení komponent kolem jednočipového počítače

Zdroj: Vlastní zpracování

Všechny napájecí a zemní piny musí být správně připojeny k napájecím zdrojům. Tyto spoje, včetně plošek, vodivých cest a proků, by měly mít co nejnižší impedanci. Toho lze obvykle dosáhnout širokými vodivými cestami.

Kromě toho by měl být každý pár napájecího napětí a země blokován filtračním kondenzátorem s hodnotou 100 nF, připojeným mezi napájecí pin STM32. Tyto kondenzátory musí být umístěny co nejbližší k odpovídajícím pinům. Typické hodnoty se pohybují od 10 nF do 100 nF. Níže uvedený obrázek ukazuje typické rozložení takového páru VDD/VSS. [8]



Obrázek č. 38 DPS / Doporučené rozložení blokovacího kondenzátoru

Zdroj: www.st.com

Na obrázku č. 37 jsou vyobrazeny dva blokovací kondenzátory. Blíže k jednočipovému počítači vidíme C6 s hodnotou 100 nF. Za ním je kondenzátor C5 s hodnotou 10 μ F. Tyto kondenzátory jsou zapojeny podle doporučení od výrobce, které je na obrázku č. 38.

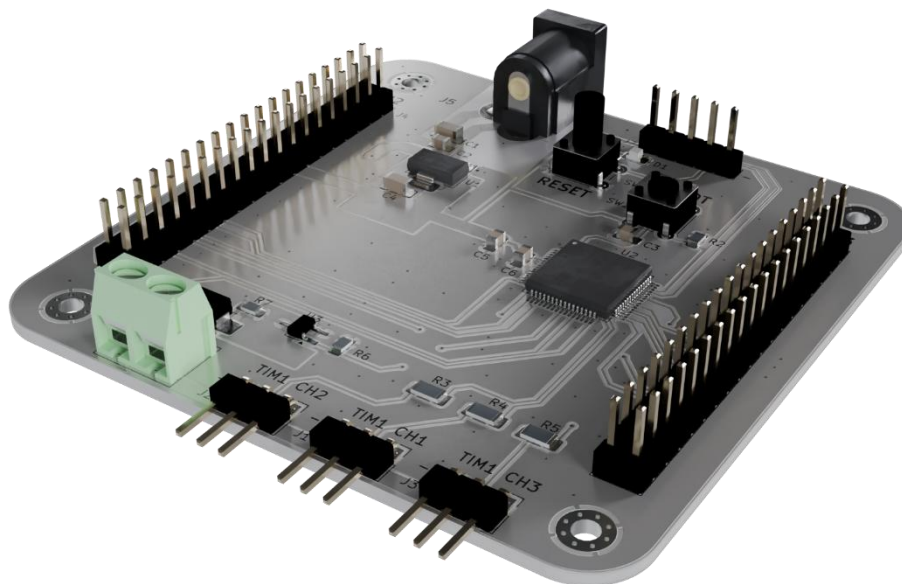
V této verzi bylo také přizpůsobeno umístění jednočipového počítače. Tak, aby většina cest, které vedou vysokorychlostní signály (SPIA, SPIB) byla co nejblíže k periférii (LCD, paměti). Tato změna přispěla k potlačení potencionálního šumu a eventuální ztrátě dat.

Napájecí zdroj byl zvolen na základě potřebného proudu. Skutečné množství energie potřebné pro Pixely závisí na mnoha faktorech a je obtížné odhadovat jejich spotřebu Technologie LED modulů používaných v Pixelech se neustále zlepšuje, ale změny ve specifikacích nejsou prodejci LED pásků jasně komunikovány.

Například najdete mnoho webových stránek a příspěvků, které s jistotou uvádějí, že spotřeba energie jednoho modulu WS2812B při plném jasu bílé (R+G+B) je 0,3 W. Protože se jedná o zařízení na 5 V, znamená to celkový odběr proudu $0,3 \text{ W} / 5 \text{ V} = 0,06 \text{ A}$ neboli 60 mA. To však platilo pro velmi staré verze WS2812B, ale u aktuální verze (WS2812B-V5) je proud při plném jasu bílé pouze 0,18 W, tedy 36 mA. [1]

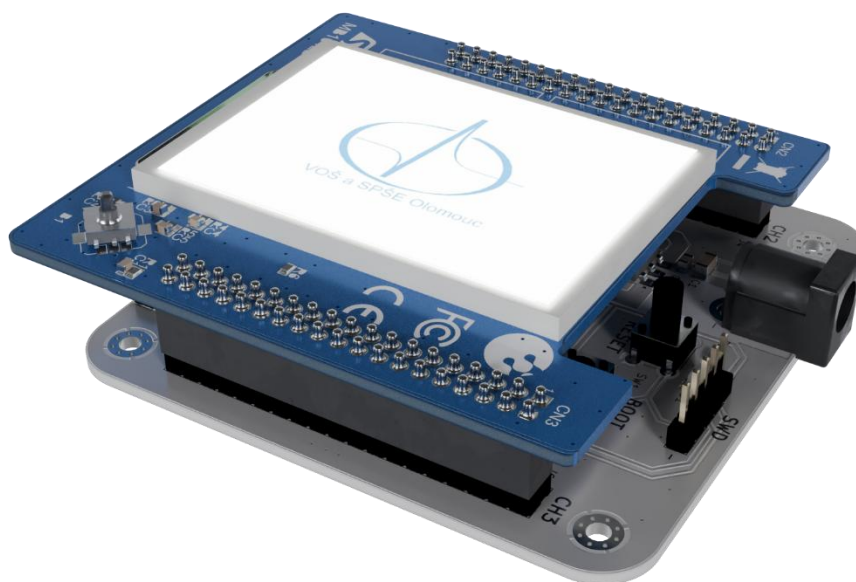
Hrací deska je složena ze 72 individuálních pixelů. To znamená, že maximální odebíraný proud může být 2,592 A. Prakticky se k této hodnotě však nikdy nedostaneme, jelikož nenastane situace, kde budou všechny Pixely svítit bíle zároveň.

Musíme také brát v potaz to, že rozšiřující modul s displejem požaduje taky několik set miliampér. Nakonec jsme tedy pro napájení zvolili spínaný zásuvkový zdroj 5 VDC 4 A. Napětí tohoto zdroje je ve skutečnosti 5,35 V což není ideální pro pixely, jejichž maximální napětí uvedené v katalogu je 5,3 V. Tento problém může způsobit kratší životnost Pixelů.



Obrázek č. 39 DPS / 3D vizualizace

Zdroj: Vlastní zpracování



Obrázek č. 40 DPS / 3D Vizualizace DPS s rozšiřujícím modulem

Zdroj: Vlastní zpracování

5. VZNIKLÉ PROBLÉMY

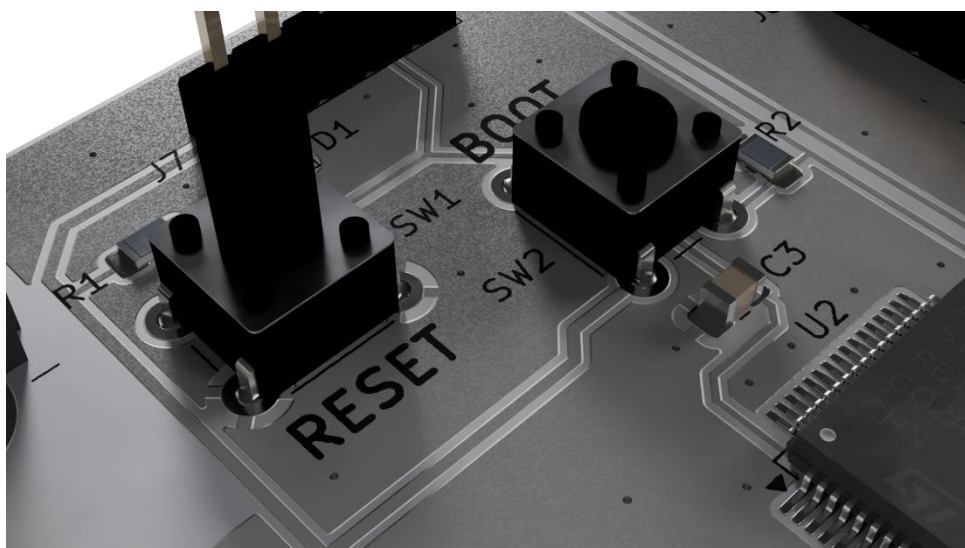
Po výrobě a zapájení jednotlivých komponent se projevilo hned několik problémů, které se musely vyřešit. Problémy však nebyly pouze hardwarové, ale i firmware obsahoval několik chyb, které bylo nutné opravit

5.1 PIN NRST

Funkce tohoto pinu je popsána v kapitole 4.1.2, a víme už tedy že proto, aby se vykonával program musí být na tomto pinu 3,3 V. Tato napěťová úroveň však nebyla stabilní. Několik minut po zapnutí systému jsme naměřili na tomto pinu 3,3 V. Po několika dalších minutách správné funkce se však na cestě NRST naměřila plovoucí napěťová úroveň. Někdy na cestě bylo 0,9 V, jindy 0,1 V. To znamenalo, že byl mikrokontroler neustále v hardwarovém resetu, nevykonával se tedy program.

První možností příčiny tohoto problému mohl být interní watchdog, nebo jiný subsystém, který umí generovat reset. Po výměně mikrokontroleru, kontrole celého firmwaru a kontrole nastavení periférií však problém přetrvával.

Další indicií byl odpor, který jsme naměřili mezi NRST a GND. Ten se pohyboval ve stovkách ohmů, což naznačovalo, že někde na DPS je zkrat. Odpor mezi těmito dvěma body se však každou sekundou zvyšoval. Tento problém také znemožnil připojení k mikrokontroleru pomocí programátoru.



Obrázek č. 41 Cesta NRST vedoucí k tlačítku SW1

Zdroj: Vlastní zpracování

Na obrázku vidíme cestu NRST, která vede z pod mikrokontroleru přes kondenzátor C3, pod tlačítkem SW2 směrem k SW1 a rozhraní pro SWD. Jediná možná příčina tohoto problému byla chyba v DPS. Pro kontrolu cesty NRST, jsme se tedy rozhodli odpájet kondenzátor C3, tlačítko SW2 a tlačítko SW1. Po odpájení tlačítka SW2 a následném proměření jsme zjistili, že na cestě NRST je požadovaná napěťová úroveň 3,3 V. Problém byl tedy s největší pravděpodobností způsoben spodní stranou tlačítka SW2. Toto tlačítko je redundantní. Vyřešení tohoto problému nám také umožnilo připojit se k mikrokontroleru pomocí programátoru. Aby tento problém nenastal znovu, třeba i z jiného důvodu, změnili jsme funkci pinu NRST na GPIO, a to pomocí option bytů v STM32CubeProgrammer. Tento preventivní krok však znemožnil provedení hardwarového resetu pomocí tlačítka SW1.

5.2 NÁHODNÉ CHOVÁNÍ PIXELŮ

Řešení tohoto nakonec banálního problému vyžadovalo několik týdnů testování a přes 3 metry LED pásku. Problém spočíval v tom, že Pixely fungovaly správně po dobu několika desítek minut, někdy i hodin, ale po náhodně dlouhém časovém intervalu se Pixely začaly chovat chaoticky. Někdy začaly náhodně blikat, jindy zase svítily první dva zeleně a zbytek nesvítil. Přestože byla povaha tohoto problému zcela náhodná, jeho replikace byla jednoduchá. Právě z tohoto důvodu jsme jako první řešili integritu datového signálu a vstupního napájení.

Integritu datového signálu jsme si ověřili pomocí osciloskopu, kde jsme na vstup prvního pixelu přidělali sondu. Signál vypadal stejně jako na obrázku č. 10, tedy dle očekávání. Na vstup putoval správný signál, i když se pixely chovaly chaoticky. To nám vyloučilo tuto možnost. Napájení Pixelů jsme taktéž otestovali pomocí osciloskopu. Tento test jsme provedli se dvěma různými napájecími zdroji a závěrem bylo to, že napájení nemá na tento problém vliv.

Jedinou zbývající možností byla tedy chyba v samotném LED pásku. Po objednání nového a následném otestování jsme byli schopni replikovat stejný problém. Problém jsme dokonce byly schopni replikovat na LED páscích od dvou různých výrobců. Díky tomuto zjištění jsme znovu obrátili pozornost na chybu na naší straně. Poslední realistickou možností, byla tedy chyba v pájení. Na jednotlivé spoje jsme se podívali pod lupou a vypadali dostačující. Následně jsme se pokusili zapájet několik

pixelů pomocí tří různých pájecích stanic, problém pořád přetrvával. Po opětovném prohlédnutí spojů jsme si všimli malých částí pájecí pasty mezi pady LED pásku. Tyto zbytky jsme pomocí isopropylalkoholu odstranili. Po několika hodinovém testování LED pásku jsme už nebyli schopni tento problém replikovat.

5.3 OSTATNÍ PROBLÉMY

V průběhu vývoje jsme narazili na několik menších problémů. Komplikace nastaly jak softwarové, tak hardwarové. Například zkrat pinů, které provádí komunikaci s externí pamětí, blikání signalizační LED, nebo problém pájení velkých ploch bez tepelných mostů

5.3.1 ZKRAT PINŮ SPI

Pro správnou funkci displeje je nutné rychle číst z externí paměti. To bylo znemožněno díky zkratu pinů SPIB_MISO a SPIB_MOSI. Tento problém jsme poprvé zaznamenali po tom, co displej nedokázal dostatečně rychle překreslovat pixely při animaci hodu kostkou. Po následném prověření všech pinů kolíkové lišty, která slouží jako rozhraní pro rozšiřující desku jsme zjistili, že právě mezi těmito dvěma piny je zkrat. Na cestě k mikrokontroleru jsme nenašli závadu. Problém jsme nakonec vyřešili tak, že jsme odpájeli a znovu připájeli mikrokontroler.

5.3.2 PÁJENÍ PLOCH BEZ TEPELNÝCH MOSTŮ

Další překážkou v procesu pájení bylo pájení rozsáhlých měděných ploch, které nebyly opatřeny tepelným můstkem. Tento problém se nejvíce projevil při pájení regulátoru napětí, jehož připojení k desce bylo realizováno cestami o šířce 2 mm. Takto široké spoje propojené s velkou měděnou plochou efektivně odváděly teplo, což komplikovalo dosažení požadované teploty pro správné pájení.

U komponent, kterých se tento problém týkal, bylo obtížné zahřát plošku na dostatečnou teplotu. Nakonec se nám podařilo regulátor napětí úspěšně zapájet, a to díky zvýšení teploty pájecí stanice na maximum.

6. SOFTWARE PRO OVLÁDÁNÍ HRY

V předchozích kapitolách bylo vysvětleno, jak hráč interaguje s hrací deskou a jak probíhá ovládání Pixelů. V této kapitole se dozvíme, jak probíhá samotná.

Hra člověče nezlob se má známá pravidla. Součástí pravidel je i možnost vlastní úpravy hry. V digitální verzi hry je několik malých úprav. Například není možné vyhodit vlastní figurky. Hra obsahuje i jiné malé úpravy, které byly provedeny za účelem zpříjemnění hráčského zážitku a zajímavější strategie. Tou největší je možnost vyhození figurky po přejetí políčka, kde se již nachází jiná figurka.

6.1 POHYB FIGUREK PO HRACÍ DESCE

Hlavní logika hry Člověče, nezlob se je implementována v souboru `clovece_nezlob_se.c`, kde klíčovou funkcí je `move_pawn`. Tato funkce je volána v `handle_in_progress` v souboru `main.c` a zajišťuje pohyb figurek na základě hodu kostkou a aktualizaci hrací desky.

Pokud hráč hodí 6, vybraná figurka se přesune ze startovní oblasti na začátek hrací desky. Počet figurek na startu se sníží a vizuální změny se provedou pomocí funkce `init_player`. Animace nasazení figurky se spustí funkcí `pawn_kick_set_board_animation`, a pokud je na daném místě jiná figurka, dojde k jejímu vyhození funkcí `kick_out_pawn`. Stav Pixelů se aktualizuje pomocí `set_LED_color`, jas se nastaví funkcí `set_brightness` (`set_brightness_individually`) a data se odešlou pomocí DMA do TIM1, který generuje PWM signál.

Pokud figurka není na startu a hráč nehodil 6, pohybuje se podle hozeného čísla. Pokud figurka dosáhne pozice 39, přesune se zpět na pozici 0. Každý krok je vizuálně zobrazen pomocí `set_LED_color` a správná pozice všech figurek je ověřena funkcí `set_position_of_all_pawns`. Funkce `check_finish_pawn` kontroluje, zda figurka může vstoupit do cílového domečku, a pokud jsou všechny figurky hráče v cíli, `check_end_game` označí hráče jako dokončeného. Mezi jednotlivými kroky pohybu figurky je nastavena prodleva 200 ms pro plynulost animace. Funkce `move_pawn` zajišťuje různé herní mechanismy, jako pohyb figurek, aktualizace Pixelů, vyhazování figurek a podmínek pro výhru.

6.2 VYHOZENÍ FIGUREK

Funkce `kick_out_pawn` zajišťuje vyhození figurek z hrací desky v případě, že se na stejné pozici ocitnou figurky různých hráčů. Tato funkce je součástí pohybové logiky a je volána v `move_pawn`.

Funkce přijímá dva argumenty: ukazatel na strukturu právě hrajícího hráče a jeho číslo. Prochází všechny hráče a kontroluje, zda je aktuální hráč stejný jako ten, který se právě pohnul. Pokud ano, přeskočí zbytek funkce. Jinak si uloží strukturu iterovaného hráče a následně projde všechny jeho figurky, jejichž pozice se dočasně uloží.

Dále se ověřuje, zda se pozice figurky iterovaného hráče shoduje s pozicí posunuté figurky, a zároveň se zkontroluje, zda figurka není v cíli, na startu nebo na startovací pozici svého hráče. Pokud některá z těchto podmínek neplatí, pokračuje se na další figurku. Pokud ale podmínky platí, figurka iterovaného hráče je vyhozena – Pixel se zhasne, na stejném místě se nastaví barva nové figurky, spustí se animace vyhození a figurka je přesunuta zpět na startovní pozici. Nakonec se aktualizují statistiky a nastaví se jas Pixelu. Výsledná data jsou odeslána do TIM1 pro generaci PWM signálu.

6.3 DOSAŽENÍ CÍLOVÉHO DOMEČKU

Funkce `move_pawn` obsahuje funkci `check_finish_pawn`. Ta určuje, jestli figurka může vstoupit do cíle. Funkce potřebuje dva argumenty: ukazatel na strukturu hrajícího hráče a číslo hrajícího hráče.

Na začátku se uloží ukazatel na pozici vybrané figurky právě hrajícího hráče. Když je její pozice rovna cílové pozici hráče +1, znamená to, že figurka může vstoupit do cílového domečku. V tomto případě se zavolá funkce `init_finish`, která tuto změnu provede vizuálně. Potom se spustí animace dokončení a zjistí se, zda jsou v cíli všechny čtyři figurky. Pokud ano, hráč vyhrál.

Funkce `init_finish` zhasne Pixel, kde figurka naposledy byla a nastaví pozici figurky do cílového domečku hráče. Pokud jsou v cíli všechny figurky, zavolá se funkce `check_player_finish`, která nastaví výslednou pozici hráče, proměnnou `is_finished` a uloží se čas dokončení do proměnné `finished_time`.

Každé kolo se volá funkce `check_end_game`, která prochází strukturu všech hráčů a inkrementuje proměnnou `num_of_finished_players`, když má iterovaný hráč nastavenou hodnotu `is_finished` na `True`. Jakmile do cíle dorazí dost (celkový počet hráčů - 1) hráčů, změní se `game_stage` ve struktuře `game_info`. Tím se hra ukončí a zobrazí se výsledky.

6.4 ANIMACE V PRŮBĚHU HRY

V průběhu hry se můžeme setkat s několika různými animacemi, které slouží k povýšení vizuálního zážitku, nebo jako zpětná vazba pro hráče. Nejdůležitější animací je ta při vybírání figurky. Slouží k tomu, aby hráč viděl, se kterou figurkou plánuje posunutí na hrací desce.

6.4.1 VÝBĚR FIGURKY | ANIMACE

Po hodu kostkou si každý hráč musí vybrat figurku, se kterou se posune na hrací desce. Figurka, která je právě vybraná začne blikat.

Tato animace se aktivuje pomocí funkce `delay`, která slouží jako rozbočovač při výběru animace. V případě výběru figurky se zavolá funkce `handle_select_pawn_delay`. Ta si načte strukturu právě hrajícího hráče. Když se vybraná figurka liší od té předchozí, musíme se ujistit, že poslední vybraný Pixel (figurka) zůstane rozsvícený. To znamená, že nastavíme jeho jas na 200.

Když pozice figurky není v cíli, ani na startu, vykonají se instrukce v následujícím odstavci. Tento stav by však nikdy neměl nastat, jelikož je řešen v logice výběru figurky. Pro jistotu byla tato podmínka implementována i sem.

Každých 500 ms zjistíme, jaký byl minulý stav Pixelu (rozsvíceno nebo zhasnuto) a na základě této informace provedeme opak. Tímto docílíme nekonečné blikající animace vybrané figurky.

6.4.2 NASAZENÍ A VYHOZENÍ FIGURKY | ANIMACE

Pokaždé, když se na hrací pole nasazuje figurka nebo je vyhozena, spustí se tato animace. Jako první si musíme uložit strukturu právě hrajícího hráče. Následně šestkrát iterujeme a inkrementujeme proměnnou `i`. Když je `i` sudé nastavíme jas pozice figurky (po nasazení na hrací desku) na 30. Musíme také zavolat funkci

pawn_kick_set_start_animation, která provede to stejné, akorát ve startovacím domečku. Tato data pošleme a proměnná *i* se inkrementuje. To znamená, že je lichá. Počkáme 250 ms a nastavíme jas jak na hrací desce, tak ve startovacím domečku na 30. Tento proces se opakuje do té doby, než hodnota proměnné *i* není větší než 7.

6.4.3 DOSAŽENÍ CÍLE | ANIMACE

Když figurka obejde celou hrací desku dostane se do cílového domečku. To spustí animaci, která mění jas Pixelů v cíli. Pro realizaci této animace je využita vnořená smyčka *for*. Ta vnější iteruje čtyřikrát. Ta vnitřní iteruje tolikrát, kolik je v cílovém domečku figurek. Jas první figurky v domečku se nastaví na 30, počká se 120 ms a jas se nastaví na 250, po opětovném čekání se tento proces opakuje pro každou figurku v domečku čtyřikrát.

6.4.4 VÝHRA HRÁČE | ANIMACE

Když kterýkoli hráč dostane všechny čtyři své figurky do cíle spustí se po animaci v kapitole 6.4.3 ještě speciální animace, která mění jas celého cílového domečku zároveň. Tento proces funguje na stejném principu, který je popsán v předchozích kapitolách, a to je iterace a následné nastavení jasu dle iterované proměnné.

6.5 NASTAVENÍ | ANIMACE

Před začátkem hry jsou hráči v menu nastavení a hrací deska není využita, proto je na hrací desce vyobrazena zajímavá animace. Její implementace se nachází ve funkcích, které jsou přístupné přes funkci *delay*.

Hrací deska je rozdělena do tří na sobě nezávislých částí, a to startovací domečky, cílové domečky a hrací deska (plocha, po které se posouvají figurky). Každá z těchto částí vykonává vlastní animaci. Jako první se podíváme na animaci hrací desky.

Každých 40 ms nastavíme jas Pixelů na indexech $i + 2$, $i + 12$, $i + 22$ a *i* hráče 4 (speciální proměnná je vyžadována, jelikož v průběhu animace přechází hodnota této proměnné z 39 na 0) na 200. Barvu na indexu $i + 2$ nastavíme na červenou, na $i + 12$ na modrou, na $i + 22$ na žlutou a na indexu proměnné pro hráče 4 na zelenou. Tímto docílíme plynulého efektu načítání.

Když se proměnná *i* inkrementuje desetkrát, znamená to, že všechny pixely na hrací desce jsou rozsvícené a musíme je postupně začít vypínat. To provedeme stejným procesem jako u rozsvěcování, akorát nastavíme jas na 0. To vytvoří plynulý efekt zpětného načítání.

Další efekt je ve všech startovacích domečkích, kde dva pixely vždy svítí, jeden s vyšším jasnem a druhý s nižším. Tyto dva pixely neustále „běží“ dokolečka. Toho docílíme tak, že si uložíme do pole indexy Pixelů v pořadí, jak se budou rozsvěcovat. Následně každých 120 ms nastavíme jas dle pole a jas předchozího Pixelu nastavíme na 0.

Na stejném principu funguje i efekt, který vytváří Pixely v cílovém domečku. Jedinou změnou je pořadí, ve kterém se pixely rozsvěcují a zhasínají. Je to provedeno tak, aby se světlo postupně přibližovalo do samotného středu a následně vyresetovalo zpět na začátek.

6.6 VÝBĚR FIGURKY ALGORITMICKY ŘÍZENÉHO HRÁČE

V nastavení je možné zapnout a vybrat počet algoritmicky řízených hráčů. Tito hráči jsou řízeni algoritmem pro výběr figurky. Tento algoritmus je primitivní a je založen na generování náhodných čísel. Algoritmus by šel vylepšit pro lepší výsledky ve hře. Po testování algoritmu založeném na generování náhodných čísel jsme však došli k závěru, že je dostačující a vytváření chytřejšího algoritmu není nutné, jelikož by hra byla těžší.

Jako první musíme náhodně vybrat figurku. Potom musíme zjistit, jestli vybraná figurka není v cíli, když je, tak ji změníme. Když je hozené číslo šest, znamená to, že můžeme nasadit figurku ze startovacího domečku (pokud tam nějaká je). Když hodíme číslo jiné, než 6 iterujeme jednotlivými figurkami a hledáme figurku, která není na startu, nebo v cíli.

7. KONSTRUKCE

Důležitou součástí výrobku je i konstrukce, která spojí jednotlivé prvky dohromady. Ta je složena z pěti částí, které jsou přišroubovány a přilepeny dohromady. 2 části slouží jako základna konstrukce, ta obsahuje kanály pro LED pásy. Další dvě části jsou kryt desky. Tyto části jsou slepené dohromady lepidlem na plast. Slouží jako hrací plocha pro Člověče nezlob se. Poslední je základna, která je důležitá pro uchycení DPS a displeje ke zbytku výrobku. Celá konstrukce byla realizována pomocí 3D tiskárny.

7.1 POPIS KONSTRUKCE

Vrchní část desky obsahuje otvory, které slouží jako dráha pohybu figurek. Pod těmito otvory se nachází samotné Pixely.

V přední části konstrukce se nachází obdélníkový výřez pro displej, který je mírně nakloněný pro lepší čitelnost. Pod krytem jde na obrázku č. 43 vidět přímo deska plošných spojů osazená jednočipovým počítačem STM32, který řídí celou hru. Připojení jednotlivých periférií je řešeno pomocí konektorů. Na boční straně konstrukce je umístěn konektor pro napájení.

Celkový design je ergonomický, se zaoblenými hranami, což přispívá k pohodlnému používání výrobku. Modulární provedení konstrukce umožňuje snadnou montáž a údržbu jednotlivých komponent.

7.2 3D TISK

3D tisk umožňuje tvorbu fyzických modelů na základě digitálních modelů. Proces je založen na postupném nanášení vrstev materiálu. Pro návrh modelu k tisku byly využity programy Autodesk Inventor a Blender. Model byl následně převeden do formátu STL a zpracován slicer softwarem PrusaSlicer, který vygeneruje instrukce pro tiskárnu.



Obrázek č. 42 3D vizualizace výrobku

Zdroj: Vlastní zpracování



Obrázek č. 43 3D vizualizace rozloženého výrobku

Zdroj: Vlastní zpracování

ZÁVĚR

SEZNÁMENÍ S WS2812B

Seznámili jsme se s vnitřní strukturou pixelů, metodou jejich ovládání a využití v praxi. Úspěšně jsme vytvořili driver pro ovládání pixelů za pomoci STM32.

Po vytvoření hrací desky z pixelů se objevilo několik hardwarových problémů, které byly špatně reprodukovatelné. Například po několika hodinách správně funkce pixelů přestane fungovat část LED pásku. Po několika týdnech testování se však problém vyřešil a WS2812B fungují dle očekávání.

HRACÍ DESKA PRO ČLOVĚČE NEZLOB SE

Hrací desku jsme realizovali pomocí LED pásku a její konstrukci pomocí 3D tisku. Součástí hrací desky je i displej s joystickem, pomocí kterého hráči interagují s figurkami. Na displeji se zobrazí animace hrací kostky, kterou jsme realizovali pomocí widgetu Animated Image. Grafické rozhraní jsme vytvořili pomocí aplikace TouchGFX Designer a logiku jsme naprogramovali pomocí GUI frameworku TouchGFX, který je napsán v jazyce C++. V práci jsme se seznámili s objektově orientovaným programováním v jazyce C++.

MOŽNOST HRY S ALGORITMICKÝMI HRÁČI

Hru jsme naprogramovali tak, aby před začátkem hry bylo možné vybrat počet skutečných a počet algoritmických hráčů. Tito hráči jsou řízeni pomocí funkce, která je založená na pseudonáhodném výběru figurky a následné akci dle hozeného čísla. Tento algoritmus lze považovat za primitivní a může mít mnoho vylepšení. Testováním jsme dospěli k závěru, že pro hru Člověče nezlob se je tento primitivní algoritmus dostačující. Možné vylepšení jako například posun figurky na základě pozice ostatních figurek by bylo možné implementovat.

NÁVRH DESKY PLOŠNÝCH SPOJŮ

DPS jsme vytvořili tak, aby obsahovala nutné prvky pro tento projekt. Důležitou součástí je rozhraní pro rozšiřující desku s displejem a rozhraní pro LED pásek. Deska obsahuje také rozhraní pro reproduktor, tato část však nebyla dokončena.

Deska byla vyrobena firmou JLCPCB. Jednotlivé komponenty jsme následně zapájeli. Opět jsme zaznamenali několik problémů. Například jsme nemohli při nahrávání programu do STM32 vymazat flash paměť displeje. Následně jsme zjistili že mezi dvěma piny pro komunikaci s pamětí byl zkrat. Dále jsme zkratovali piny VSS a VDD, čímž jsme rozbili jeden mikrokontroler.

NÁVRHY NA VYLEPŠENÍ

V průběhu projektu jsme dospěli k mnoha různým vylepšením, které by mohl výrobek obsahovat. Některé jsme implementovali a na jiné nezbyl čas. Například audio zpětná vazba, která by zvýšila interaktivitu hry. Algoritmus pro ovládání strojových hráčů má také mnoho prostoru pro vylepšení. V průběhu projektu jsme zjistili, že některé STM32 obsahují periferii RNG (Random Number Generator), která by vylepšila způsob generování náhodných čísel.

SEZNAM POUŽITÉ LITERATURY

- [1] **aerokieth**. *Wiring Design for Addressable LED Strips*. Online. Electric Fire Design. 2022, s. 7. Dostupné z: <https://electricfiredesign.com/2022/04/14/wiring-design-for-addressable-led-strips/> [cit. 28.2.2025].
- [2] *Human Vision and Color Perception*. Online. EVIDENT. S. 13. Dostupné z: <https://evidentscientific.com/en/microscope-resource/knowledge-hub/lightandcolor/humanvisionintro> [cit. 22.3.2025].
- [3] **Křížan V.** *EPO-07-Paměti*. Prezentace prezentována v: [Vyučovací hodina Elektronických počítačů SPŠEOL; 3.12.2024, Olomouc, Czechia].
- [4] **Křížan V.** *EPO-08-HDD*. Prezentace prezentována v: [Vyučovací hodina Elektronických počítačů SPŠEOL; 14.1.2025, Olomouc, Czechia].
- [5] **Křížan V.** *EPO-11-Monitory*. Prezentace prezentována v: [Vyučovací hodina Elektronických počítačů SPŠEOL; 20.3.2025, Olomouc, Czechia].
- [6] **Macronix** [Online katalogový list]. *MX25L6433F*. ©2023 [cit. 23.3.2025]. Dostupné z: https://eu.mouser.com/datasheet/2/819/MX25L6433F_2c_3V_2c_64Mb_2c_v1_8-3371028.pdf.
- [7] **NOR vs. NAND Flash Memory**. In: Youtube [online]. 10. 2. 2021 [cit. 23.3.2025]. Dostupné z: https://www.youtube.com/watch?v=jAQiMWmSIlo&ab_channel=EyeonTech.
- [8] **STMicroelectronics** [Online aplikační poznámka]. *AN5096 Application note Getting started with STM32G0 Series hardware development*. ©2021 [cit. 21.3.2025]. Dostupné z: https://www.st.com/resource/en/application_note/an5096-getting-started-with-stm32g0-series-hardware-development-stmicroelectronics.pdf.
- [9] **STMicroelectronics** [Online katalogový list]. *LD1117*. ©2020 [cit. 19.3.2025]. Dostupné z: <https://www.st.com/resource/en/datasheet/ld1117.pdf>.
- [10] **STMicroelectronics** [Online katalogový list]. *STM32G071x8/xB*. ©2021 [cit. 19.3.2025]. Dostupné z: <https://www.st.com/en/microcontrollers-microprocessors/stm32g071rb.html>.

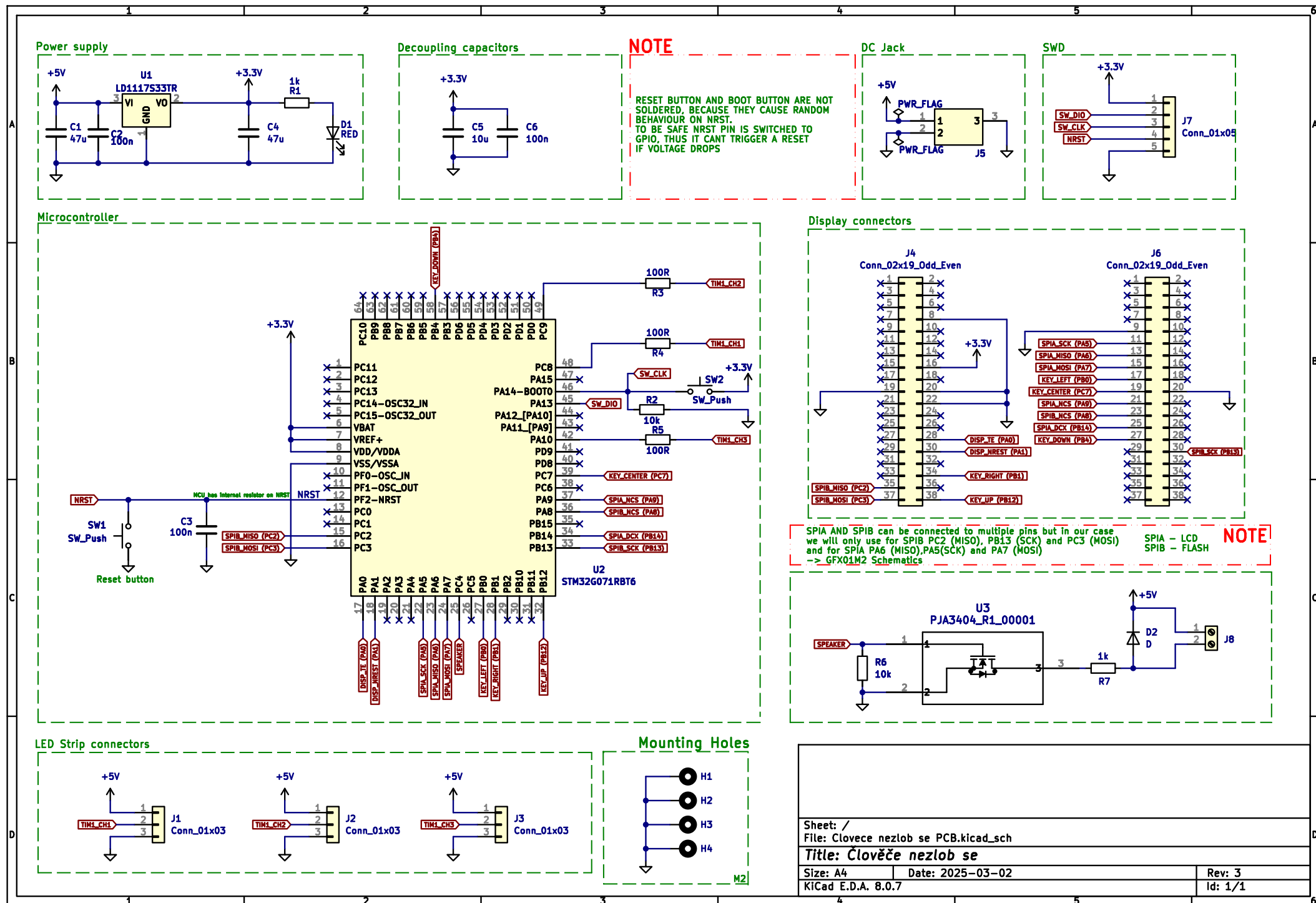
- [11] **STMicroelectronics** [Online katalogový list]. *X-NUCLEO-GFX01M2*. ©2021 [cit. 27.3.2025]. Dostupné z: https://www.st.com/resource/en/data_brief/x-nucleo-gfx01m2.pdf.
- [12] **STMicroelectronics** [Online referenční manuál]. *RM0444 Reference manual STM32G0x1 advanced Arm®-based 32-bit MCUs*. ©2024 [cit. 21.3.2025]. Dostupné z: https://www.st.com/resource/en/reference_manual/rm0444-stm32g0x1-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.
- [13] **STMicroelectronics**. *Getting started with SPI*. Online. STM32 Arm® Cortex® MCU wiki. 12 April 2024. Dostupné z: https://wiki.st.com/stm32mcu/wiki/Getting_started_with_SPI#What_is_Serial_Peripheral_Interface_-28SPI-29-- [cit. 23.3.2025].
- [14] **What is TouchGFX?** Online. *TouchGFX Documentation*. S. 5. Dostupné z: <https://support.touchgfx.com/docs/introduction/what-is-touchgfx> [cit. 24.3.2025].
- [15] **Worldsemi** [Online katalogový list]. *WS2812B*. [cit. 21.3.2025]. Dostupné z: <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf>.
- [16] **Worldsemi** [Online katalogový list]. *WS2812B-V5*. [cit. 21.3.2025]. Dostupné z: https://www.peace-corp.co.jp/data/WS2812B-V5_V1.0_EN.pdf.
- [17] **How Does Flash Memory Work? (SSD)**. In: Youtube [online]. 23. 7. 2020 [cit. 23.3.2025]. Dostupné z: https://www.youtube.com/watch?v=r2KaVfSH884&ab_channel=BLITZ.

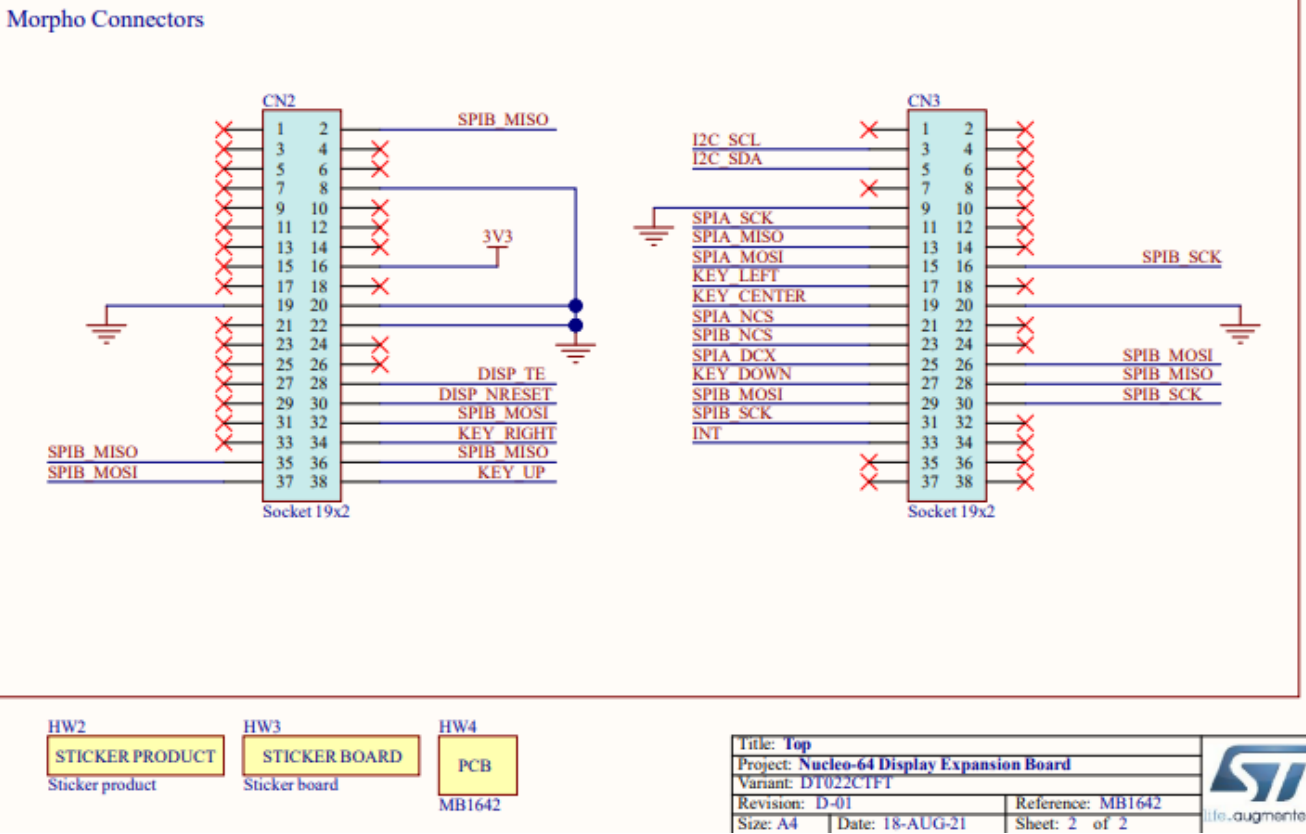
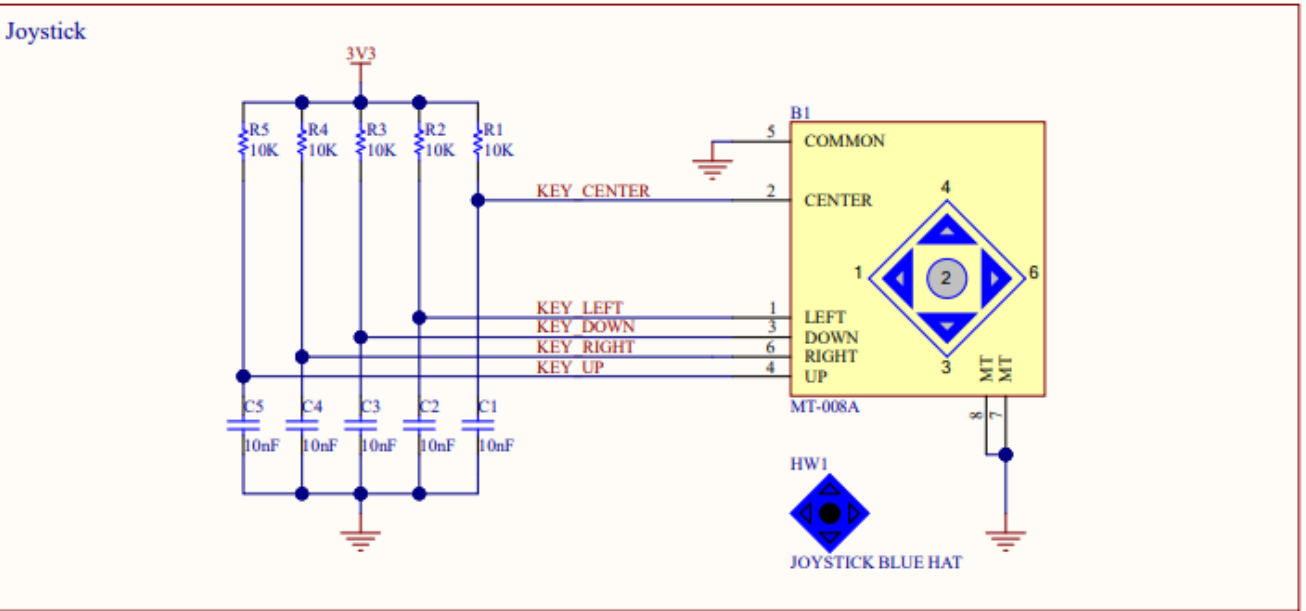
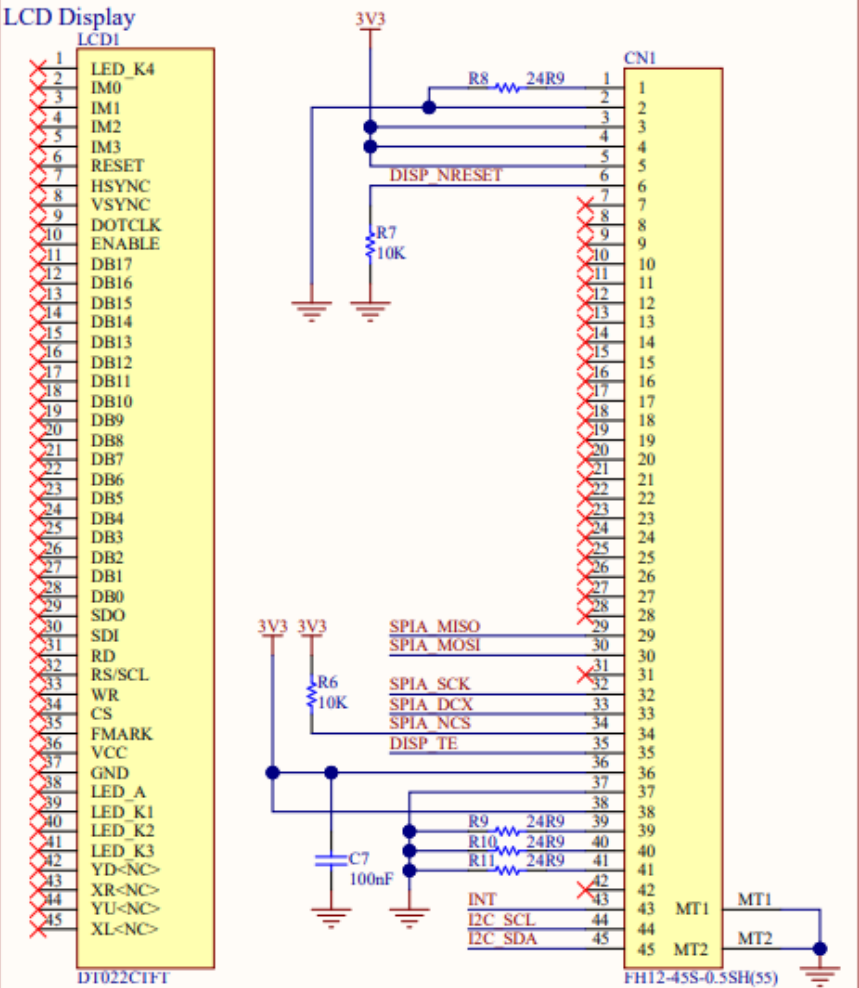
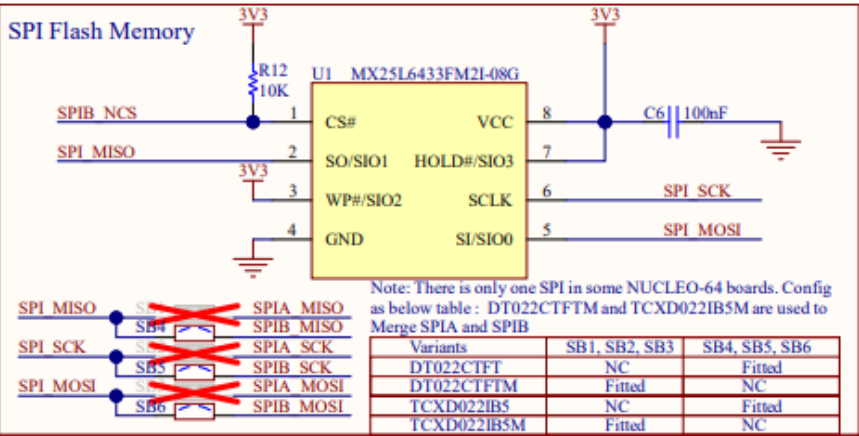
SEZNAM OBRÁZKŮ

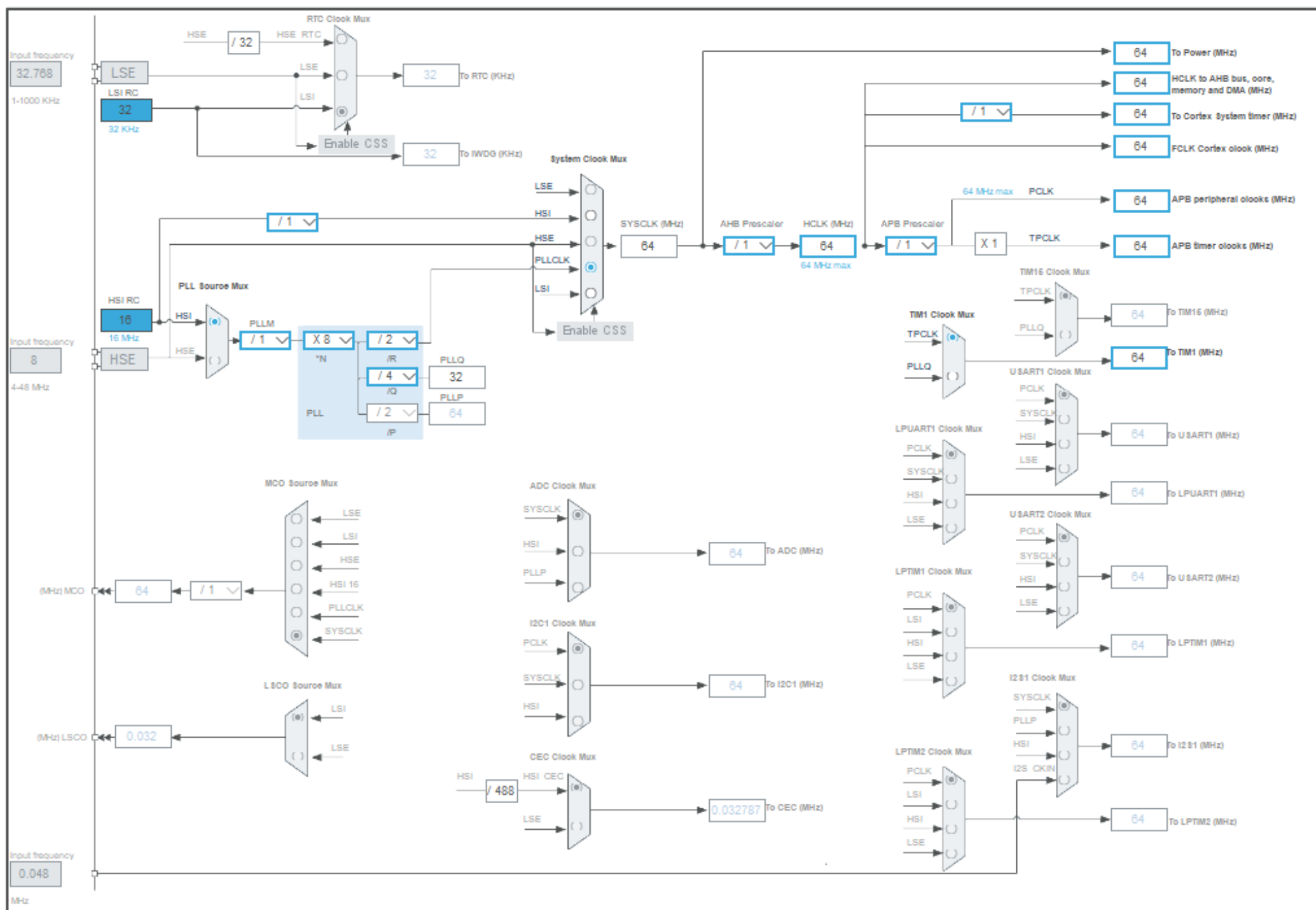
Obrázek č. 1 Blokové schéma.....	8
Obrázek č. 2 WS2812B Vnitřní struktura	10
Obrázek č. 3 WS2812B Požadavky pro datový signál.....	11
Obrázek č. 4 TIM1 Blokové schéma	12
Obrázek č. 5 TIM1 Nastavení hodin	13
Obrázek č. 6 WS2812B Uspořádání 24b informace.....	14
Obrázek č. 7 DMA Blokový diagram DMA kontroléru	15
Obrázek č. 8 DMA Blokové schéma využití DMA v systému.....	16
Obrázek č. 9 WS2812B Graf měřítkového faktoru pro nastavení jasu.....	17
Obrázek č. 10 WS2812B Skutečný datový signál.....	18
<i>Obrázek č. 11 X-NUCLEO-GFX01M2</i>	<i>19</i>
Obrázek č. 12 LCD Princip funkce LCD displeje	20
Obrázek č. 13 LCD Konstrukce barevného LCD	20
Obrázek č. 14 LCD Thin Film Transistor u jednotlivých subpixelů	21
Obrázek č. 15 LCD Jednotlivé vrstvy displeje.....	21
Obrázek č. 16 Flash Floating gate tranzistor	22
Obrázek č. 17 Flash NOR vs. NAND Flash	24
Obrázek č. 18 Flash MX25L6433FM2I-08G	24
Obrázek č. 19 SPI Full-duplex komunikace.....	25
Obrázek č. 20 SPI Diagram toku dat	26
Obrázek č. 21 SPI Čtení z flash paměti	27
Obrázek č. 22 TouchGFX Logo.....	28
Obrázek č. 23 TouchGFX numberOfAllPlayersSetting.....	28
Obrázek č. 24 TouchGFX Widgety fillWithBotsSetting, numberOfAIPlayersSetting, numberOfRealPlayersSetting a startGame.....	30
Obrázek č. 25 TouchGFX Výsledek hodu kostkou	31
Obrázek č. 26 TouchGFX Obrazovka 3	31
Obrázek č. 27 TouchGFX Obrazovka 4	32
Obrázek č. 28 TouchGFX playerResultContainer.....	33
Obrázek č. 29 TouchGFX Obrazovka 5	34
Obrázek č. 30 TouchGFX Nastavení TouchGFX Generator.....	35

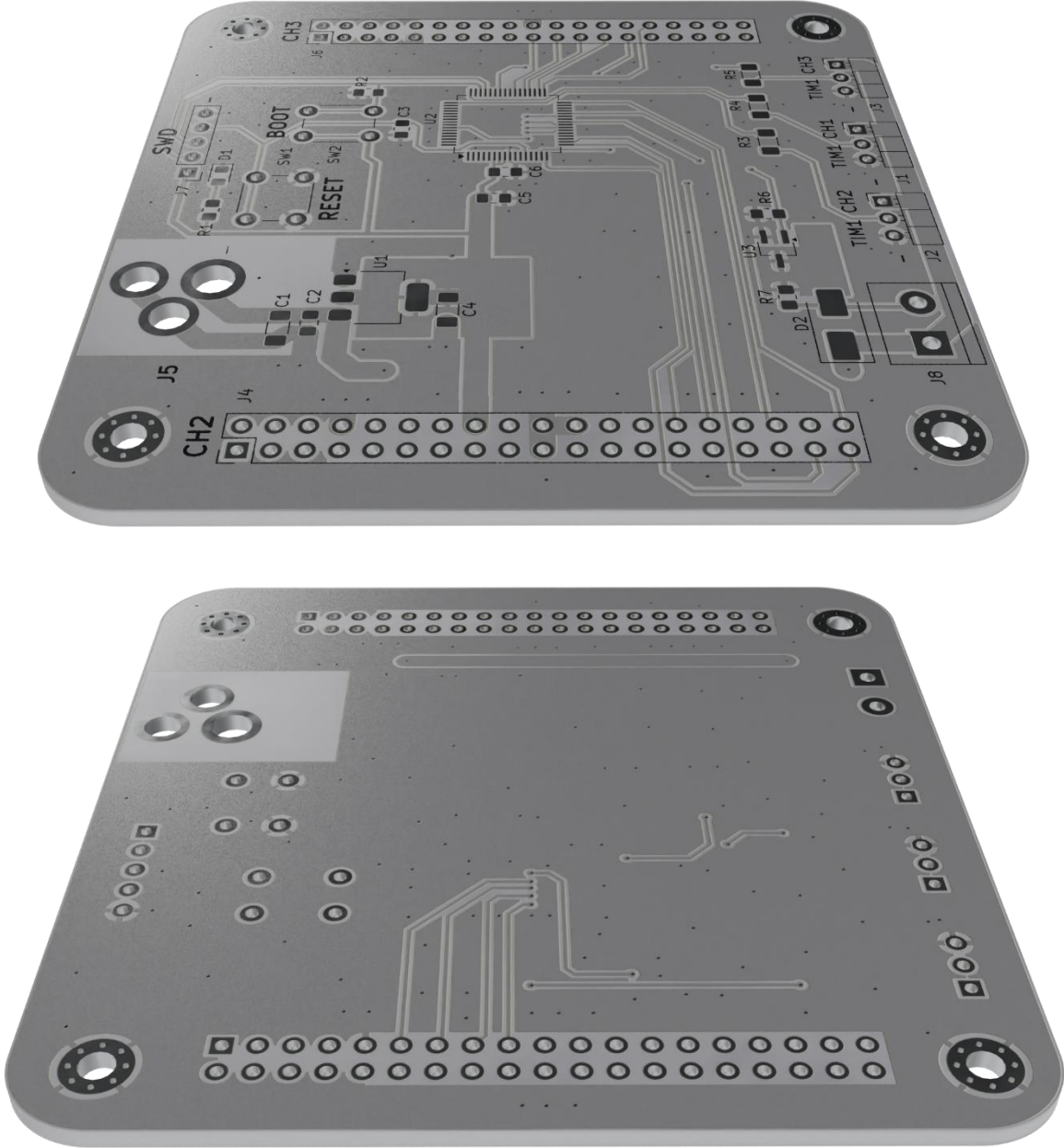
Obrázek č. 31 TouchGFX Obrazovka 6	35
Obrázek č. 32 LDO Regulátor napětí.....	37
Obrázek č. 33 STM32 Zapojení jednočipového počítače.....	39
Obrázek č. 34 X-NUCLEO-GFX01M2 Rozhraní pro rozšiřující modul	40
Obrázek č. 35 Rozhraní pro připojení LED pásků	40
Obrázek č. 36 DPS Termální podložka u regulátoru napětí.....	42
Obrázek č. 37 DPS Rozložení komponent kolem jednočipového počítače	43
Obrázek č. 38 DPS Doporučené rozložení blokovacího kondenzátoru	44
Obrázek č. 39 DPS 3D vizualizace	45
Obrázek č. 40 DPS 3D Vizualizace DPS s rozšiřujícím modulem	45
Obrázek č. 41 Cesta NRST vedoucí k tlačítku SW1	46
Obrázek č. 42 3D vizualizace výrobku.....	55
Obrázek č. 43 3D vizualizace rozloženého výrobku.....	55
 Tabulka č. 1 Flash Stavová tabulka SLC.....	 23
Tabulka č. 2 Flash Stavová tabulka MLC	23

PŘÍLOHY





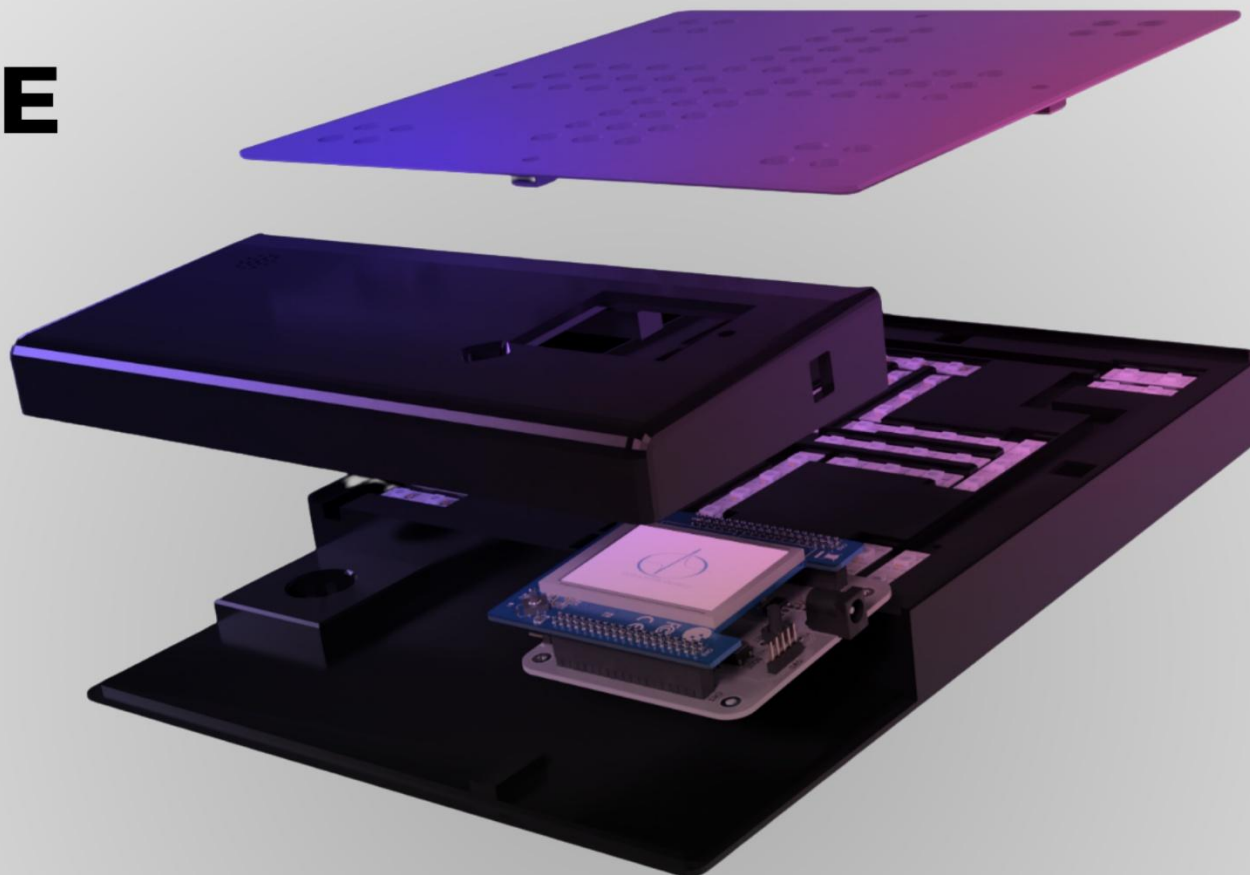




ČLOVĚČE NEZLOB SE

Příloha č. 5 Poster

VYPRACOVAL ADAM ZATLOUKAL
POD VEDENÍM ING. HELENY BLAŽKOVÉ



RGB LED
PÁSEK

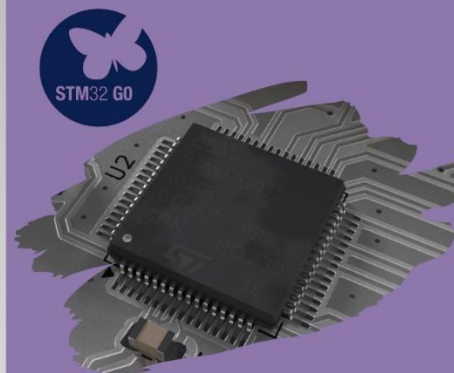


LCD DISPLEJ



 | TouchGFX

STM32



PRO VÍCE
INFORMACÍ

2025

SPŠE OLOMOUC

